Centro Universitário Senac - CAS

Campus Santo Amaro

Matheus Eiji Moriya dos Santos

Ygor Kaique de Souza Pinto

Modelagem computacional de análise de expressões faciais.

São Paulo

2022

Matheus Eiji Moriya dos Santos

Ygor Kaique de Souza Pinto

Modelagem computacional de análise de expressões faciais

Trabalho de Conclusão de Curso de Graduação apresentado ao Centro Universitário Senac - CAS (Campus Santo Amaro) como requisito parcial para a obtenção do título de Bacharel em Ciência da Computação.

Orientador(a): Mario Leandro Pires Toledo

São Paulo

2022

Resumo

Neste trabalho foi utilizado aprendizado de máquina para realizar o reconhecimento de expressões faciais e determinar a emoção do usuário que está sendo analisado. Com a evolução da tecnologia, uma nova área da computação denominada de “Computação afetiva” vem crescendo e sendo utilizada em diversos setores. Ela se baseia em diversas áreas de conhecimento, como Informática, Educação, Psicologia, Sociologia, Inteligência Artificial, entre outros.

Utilizamos da Inteligência Artificial para realizar um estudo, com o objetivo de verificar a eficácia de um sistema que identifica as emoções do usuário enquanto assiste algum vídeo. Esse estudo vem de encontro com a evolução das estratégias de propaganda direcionada, que vêm sendo realizadas pelas agências de publicidade e marketing com base nos dados coletados nas redes sociais.

As tecnologias utilizadas para o desenvolvimento do projeto foram escolhidas com base em artigos semelhantes e na análise prática de outras tecnologias que não atenderam às expectativas.

A solução é composta por um *framework* principal: *Tensorflow*, utilizado como base no treinamento de modelos de redes neurais profundas. Durante a execução dos treinamentos foram realizadas variações nas configurações e funções, com o objetivo de avaliar e alcançar a maior acurácia possível. Usamos o *Angular* para o desenvolvimento do *front-end*, em que um vídeo do *YouTube* é embutido e, enquanto o usuário assiste, é gravado com a sua câmera, analisadas as expressões faciais e definida a emoção do usuário.

Com o resultado que chegamos, foi possível concluir que, com determinadas condições de treinamento da inteligência artificial, podemos chegar em resultados expressivos de desempenho no reconhecimento de emoções do usuário.

Palavras-chave: aprendizado de máquina; emoção; computação afetiva; publicidade; marketing; inteligência artificial; *Tensorflow*

Abstract

In this work, machine learning was used to recognize facial expressions and determine the emotion of the user being analyzed. With the evolution of technology, a new area of computing called "Affective Computing" has been growing and being used in several sectors. It is based on several areas of knowledge, such as Informatics, Education, Psychology, Sociology, Artificial Intelligence, among others.

We used Artificial Intelligence to carry out a study, with the objective of verifying the effectiveness of a system that identifies the user's emotions while watching a video. This study is in line with the evolution of targeted advertising strategies, which have been carried out by advertising and marketing agencies based on data collected on social networks.

The technologies used for the development of the project were chosen based on similar articles and on the practical analysis of other technologies that did not meet expectations.

The solution consists of a main framework: Tensorflow, used as a basis for training deep neural network models, while the training occurred there were some changes to the settings and used activation functions, with the aim of evaluating and achieving the highest possible accuracy. We use Angular as our front-end, which has a YouTube video embedded and while the user watches it, we record it with his camera, analyze his facial expressions and define his emotions.

With the result we got, it was possible to conclude that with certain conditions of artificial intelligence training, we can reach expressive performance results in the recognition of user emotions.

Key-words: machine learning; emotion; affective computing; advertising; marketing; artificial intelligence; Tensorflow

Lista de figuras

Figura 1 – Posição facial em que se encontra cada uma das UAs.....15

Figura 2 – Relação entre inteligência artificial, aprendizado de máquina e aprendizado profundo……17

Figura 3 – Tipos de aprendizado de máquina.....18

Figura 4 – Exemplo de retropropagação.…….……21

Figura 5 – Exemplo de um neurônio artificial…….....23

Figura 6 – Representação matemática de um neurônio artificial…….……..23

Figura 7 – Exemplo de método de detecção facial ……...30

Figura 8 – Exemplo da geração da matriz de imagem integral.....32

Figura 9 – Comparação com a imagem original usando Haar-Like…….....32

Figura 10 – Exemplo de classificadores em cadeia ….....33

Figura 11 – Funcionamento MTCNN…..…….....34

Figura 12 – Conjunto de equações……..…….....35

Figura 13 – Equação na forma matricial.....36

Figura 14 – Técnica de estimativa de fluxo-óptico…….....37

Figura 15 – Características de Haar …….....38

Figura 16 – Média de intensidade em tons de cinza.....39

Figura 17 – Performance da análise de reconhecimento facial e acústico combinados….....42

Figura 18 – Precisão do trabalho…….....43

Figura 19 - Código de detecção facial…….....45

Figura 20 - Resultado da detecção facial …….....45

Figura 21 - Criação de massas de treinamento e validação……47

Figura 22 - Criação da camada de convolução para entrada de dados……...47

Figura 23 - Criação da camada de normalização……48

Figura 24 - Criação das camadas auxiliares da entrada…….…….48

Figura 25 - Código referente a entrada de dados…….……..48

Figura 26 - Adicionais camadas de convolução……49

Figura 27 - Criação da camada densa……...49

Figura 28 - Adição de novas camadas densas…….…..49

Figura 29 - Adição da camada Softmax…….…50

Figura 30 - Compilando a configuração da rede neural……...50

Figura 31 - Configuração da rede neural……..51

Figura 32 - Executando o treinamento da rede neural……..52

Figura 33 - Salvando o modelo resultante do treinamento….……..…52

Figura 34 - Variação batch size e número de epochs…...…….……53

Figura 35 - Variação da função de ativação…….……53

Figura 36 - Variação de otimizadores…….……54

Figura 37 - Acurácia dos modelos utilizando ReLU..…….……54

Figura 38 - Acurácia dos modelos utilizando Sigmoide...……55

Figura 39 - Acurácia dos modelos utilizando Tanh……56

Figura 40 - Acurácia por função de ativação……..……57

Lista de abreviações

EAD: Educação a Distância

FACS: *Facial Action Coding System*

UA: Unidade de Ação

ReLU: *Rectified Linear Unit*

Tanh: Tangente

SGD: *Stochastic gradient descent*

RmsPROP: *Root Mean Square Propagation*

XLA: *Accelerated Linear Algebra*

CPU: *Central Processing Unit*

GPU: *Graphics Processing Unit*

TCU: *Transmission Control Unit*

API: *Application Programming Interface*

OpenCV: *Open Source Computer Vision Library*

OpenBR: *Open Source Biometric Recognition*

MTCNN: *Multitask Cascaded Convolutional Networks*

CNN: *Convolutional Neural Network*

P-NET: *Proposal Network*

R-NET: *Refine Network*

O-NET: *Output Network*

Sumário

1 Introdução **10**
1.1 Objetivo 11
2 Revisão Bibliográfica **13**
2.1 Expressões Faciais 13
2.2 Emoção 14
2.3 Análise de Expressão Facial 14
2.3.1 FACS 15
2.4 Inteligência Artificial 16
2.5 Aprendizado de Máquina 17
2.6 Aprendizado Profundo 20
2.6.1 Redes Neurais 22
2.6.2 Funções de Ativação 24
2.6.2.1 Sigmóide 24
2.6.2.2 Tanh 24
2.6.2.3 ReLU 25
2.6.2.4 SoftMax 25
2.6.3 Otimizador 26
2.6.3.1 Adam 26
2.6.3.2 SGD 26
2.6.3.3 RMSProp 27
2.6.3.4 Adagrad 27
2.6.4 Batch Size 28
2.6.5 TensorFlow 28
2.7 Detecção Facial 29
2.7.1 Algoritmo Viola-Jones 31
2.7.2 MTCNN 33
2.7.3 Lucas-Kanade 35
2.7.4 Haar-Cascade 37
2.8 Análise Técnica 40
2.8.1 Acurácia 41
2.9 Trabalhos relacionados 41
2.9.1 Recognition of emotions from video using acoustic and facial features 41
2.9.2 RedFace – Um Sistema de Reconhecimento de Expressões Faciais para Apoiar um Ambiente Virtual de Aprendizagem. 42
2.9.3 Detecção de expressões faciais: Uma abordagem baseada em análise do fluxo óptico. 42
3 Desenvolvimento **44**

3.1 Solução proposta 44
3.2 Detecção Facial 44
3.3 Massa de dados 46
3.4 Configuração da rede neural 47
3.5 Treinamento da rede neural 52
3.6 Resultados 54

4 Conclusão 59
4.1 Trabalhos Futuros 60

5 Referências 62

1 Introdução

A comunicação e suas estratégias sempre estiveram relacionadas com inovações da tecnologia. Nos últimos anos, o poder de processamento e a expansão das redes de computadores têm aumentado significativamente. Segundo a Lei de Moore, o poder de processamento dobra a cada dezoito meses, e isso permite a criação de novos espaços e meios de comunicação, além de permitir a utilização de sistemas para compreender a reação e medir o interesse do público. Um exemplo disso é a área da publicidade e marketing, onde a tecnologia pode ser usada para estudar o comportamento do consumidor (HECK; KÜNAS; PADOIN, 2020).

No Brasil, são 171,5 milhões de usuários ativos nas redes sociais, o que representa 79,9% da população brasileira. Esse número representa um crescimento de 14,3% ou de 21 milhões de usuários de 2021 para 2022 (RODRIGUES, 2022). O crescimento recente das redes sociais vem criando muitos setores no mercado em busca de conteúdo personalizado, que se adapta de acordo com os gostos do usuário. Baseado em técnicas de extração e análise de dados, como por exemplo, likes, comentários, tempo de visualização, localização e histórico de navegação, é possível determinar o perfil do usuário e suas preferências. Essas informações são coletadas por agências de marketing e utilizadas por sistemas para realizar o direcionamento de conteúdos mais compatíveis com o perfil do cliente e suas preferências (OLIVEIRA, 2022).

O consumidor atualmente tem o poder e se tornou o centro das atenções das empresas, e, com um mercado repleto de ofertas, as empresas precisam encontrar uma forma de se diferenciar das concorrentes, para assim conseguir conquistar a confiança dos clientes. O crescimento tecnológico e das redes permite mais acesso a informações, ofertas e possibilidades de escolha, é o principal responsável pelo poder do consumidor.

Os seres humanos são influenciados e, algumas vezes, até controlados pelas emoções, tendo em vista isso, o apelo emocional se tornou uma estratégia bem sucedida pelas empresas de publicidade (LOPES, 2015).

No estágio atual da tecnologia da informação, é possível utilizar sistemas para determinar as emoções do usuário enquanto ele visualiza algo, e, a partir disso,

gerar um feedback. Esse campo de pesquisa tem recebido o nome de “Computação Afetiva”, termo criado pela pesquisadora Rosalind Picard (1997) em referência a tendência de o computador conseguir entender o estado emocional do usuário.

Uma das áreas de pesquisa que a computação afetiva abrange é a da inteligência artificial. Ela tem como objetivo possibilitar que os computadores tenham emoções, de forma que passem a conseguir ter uma interação com os usuários. Com isso, há a possibilidade do computador se tornar capaz de detectar as emoções do usuário. A computação afetiva pode ser utilizada em outras áreas, como por exemplo, na área de monitoramento, auxiliando a detectar padrões comportamentais suspeitos; a área médica, auxiliando na deteçcao de doencas por meio de sensores; a área de entretenimento, analisando as respostas de acordo com os servicos oferecidos; a área de educação, analisando os sentimentos dos alunos de acordo com uma materia específica, dentre outros (MUNIZ; RODRIGUES; GUEDES, 2016).

1.1 Objetivo

O objetivo deste trabalho é fazer um estudo sobre a modelagem de um sistema inteligente, a partir de métodos e técnicas de aprendizado de máquina, que seja capaz de realizar a detecção facial e classificação de emoções em um vídeo. Em seguida, analisar as expressões para determinar a emoção da pessoa retratada. Para atingir esse objetivo, foram utilizados algoritmos e bibliotecas especializadas no reconhecimento de face humana, como por exemplo, *Tensor Flow* e *Haar Cascade*, aliados ao aprendizado de máquina para determinar o tipo de emoção que está sendo apresentada pelo ser humano. Os algoritmos de detecção possuem a função de fornecer dados para nossa máquina, onde foram feitas as análises destes dados e, em seguida, ocorreu, também, o aprendizado da máquina.

O projeto proposto busca detectar as emoções a partir da face humana e determinar o estado emocional do usuário, com o intuito de coletar feedback do usuário a respeito da informação que está sendo apresentada a ele. Esse tipo de tecnologia pode ser usado em diversas áreas, como por exemplo, na área do marketing para testar as emoções do cliente a determinado produto ou propaganda,

na área da saúde para avaliar as reações do paciente durante uma sessão de terapia psicológica ou até em uma aula EAD para ver o que os alunos sentem durante a aula.

2 Revisão Bibliográfica

2.1 Expressões Faciais

O rosto é a parte mais marcante de um ser humano durante relações sociais entre humanos, devido ao seu potencial comunicativo, informativo e ao foco do ser humano nos rostos durante as interações. A partir dele, inferimos e adquirimos muitas informações sobre o indivíduo como sua idade, etnia e gênero (LIBRALON, 2014).

Sabe-se que o ser humano utiliza o rosto como um regulador das conversações, abrindo e fechando canais de comunicação (por exemplo, virando o rosto e desviando o olhar), complementando e qualificando outros sinais não verbais emitidos pelo indivíduo (quando dizemos "estou triste" e choramos), e em substituição a mensagens verbais, como por exemplo, quando alguém nos pergunta "gostou" e sorrimos, ao invés de falar (SILVA; SILVA, 1995).

A partir da face humana, conseguimos extrair um alto volume de informações a respeito do indivíduo, através de vários tipos de sinais apresentados nela. Paul Ekman, um dos principais pesquisadores de expressões faciais, os classifica em quatro tipos, sendo eles: (EKMAN, 1973).

a. Sinais estáticos: não mudam ou mudam pouco durante toda a vida do indivíduo, por exemplo, a estrutura óssea e a pigmentação da pele.
b. Sinais lentos: características que mudam com o envelhecer do indivíduo e se tornam mais evidentes com o passar do tempo, por exemplo, rugas e cabelos.
c. Sinais rápidos: aqueles que ocorrem por um curto período de tempo, e podem ser mais sutis ou não, por exemplo, suor, contrações musculares, posição da cabeça, coloração da pele.
d. Sinais artificiais: estes interferem na propagação dos sinais estáticos e lentos, geralmente, utilizados para enaltecer a beleza ou combater os sinais da idade do indivíduo, por exemplo, cosméticos e tinturas.

2.2 Emoção

As emoções estão presentes na vida de todos os seres humanos. Elas podem estar relacionadas à convivência em sociedade, aos conflitos entre expectativa e realidade, a mecanismos de autopreservação ou da presença de consciência (LIBRALON, 2014). São padrões de reações fisiológicas identificados através de expressões corporais, como movimento corporal, gestos, expressões faciais, entonação da voz, etc (MUNIZ; RODRIGUES; GUEDES, 2016).

É fundamental para a experiência humana a presença das emoções, pois influencia em diversos tipos de comportamentos, como a cognição, percepção, e em tarefas diárias, tais como comunicação e aprendizado. Porém, um mesmo estado emocional pode ser considerado bom ou ruim dependendo da situação em que ele ocorre, como a ansiedade, que antes do nascimento de um filho pode ser visto como um sentimento bom, enquanto a ansiedade antes de uma apresentação em público pode ser considerada como algo ruim (MUNIZ; RODRIGUES; GUEDES, 2016).

É importante não confundir emoção com o humor. Enquanto a emoção representa um sentimento mais espontâneo após determinado estímulo, o humor representa estados mais duradouros e disseminados (LIBRALON, Giampaolo, 2014).

Na visão computacional, a compreensão das emoções representa principalmente a habilidade de um computador não apenas de detectar a informação emocional, mas também de analisar, processar, construir e, com isso, montar um modelo computacional do usuário (LIBRALON, 2014).

2.3 Análise de Expressão Facial

A face é um dos meios mais poderosos de comunicação não-verbal (TORRE, COHN, 2011). Suas expressões dão indícios da emoção, intenção, dor e personalidade de um indivíduo, além de regular o comportamento interpessoal.

Este tipo de análise tem grande importância para podermos analisar e entender nós humanos, suas ações e seus comportamentos em geral. Aplicações de análise de expressão facial incluem marketing, interfaces de usuário perceptivas,

interação humano-robô, detecção de motorista sonolento, tele-enfermagem, avaliação da dor, análise da interação mãe-bebê, Transtorno do Espectro Autista, robótica social, facial animação e mapeamento de expressão para jogos, entre outros.

2.3.1 FACS

O FACS (*Facial Action Coding System*, ou Sistema de Codificação de Ações Faciais) é um modelo que foi criado pelos psicólogos Paul Ekman e Wallace V. Friesen (EKMAN; FRIESEN, 1971). Segundo Ekman e Friesen, existem seis emoções básicas que são possíveis de serem identificadas com as expressões faciais: alegria, tristeza, surpresa, desgosto, raiva e medo. Porém, essa forma de classificar as emoções se tornou muito limitada, já que é comum o ser humano demonstrar mais de uma emoção ao mesmo tempo, o que resulta numa mistura de expressões emocionais básicas. Para resolver esse problema, Ekman e Friesen criaram o FACS, que é capaz de identificar as emoções baseadas nas contrações musculares faciais. Esse sistema se tornou uma das principais referências em reconhecimento de expressões faciais (DINIZ, 2013).

Os movimentos musculares são representados pelas UAs (Unidades de Ação), que podem aparecer de forma individual ou em conjunto. O FACS é composto por 32 UAs, que estão relacionadas anatomicamente com os músculos faciais. Na Figura 1, é possível ver onde se encontra cada uma delas.

Figura 1 - Posição facial em que se encontra cada uma das UAs.

UA	Músculos	UA	Músculos
1	Frontalis (central)	2	Frontalis (lateral)
4	Corrugador do supercílio, Prócero	5	Levantador da pálpebra
6	Orbicular do olho (parte orbital)	7	Orbicular do olho (parte palpebral)
8	Orbicular da boca	9	Levantador do lábio superior e asa do nariz
10	Levantador do lábio superior	11	Zigomático menor
12	Zigomático maior, Levantador do ângulo da boca	13	Canino, levantador do ângulo da boca
14	Bucinador	15	Depressor do ângulo da boca (triangular)
16	Depressor do lábio inferior	17	Mental
18	Incisivo do lábio	20	Risório, Platisma
22	Orbicular da boca (parte externa), Bucinador	23	Orbicular da boca (parte interna)
24	Orbicular da boca (parte interna)	25	Depressor do lábio inferior, mental, Orbicular da boca
26	Temporal, Masseter	27	Digástrico, temporal, masseter
28	Orbicular da boca	38	Nasal (parte alar)
39	Nasal (parte transversal), Depressor do septo nasal	43	Orbicular do olho
45	Levantador da pálpebra, orbicular do olho	46	Orbicular do olho

Fonte: LIBRALON (2014).

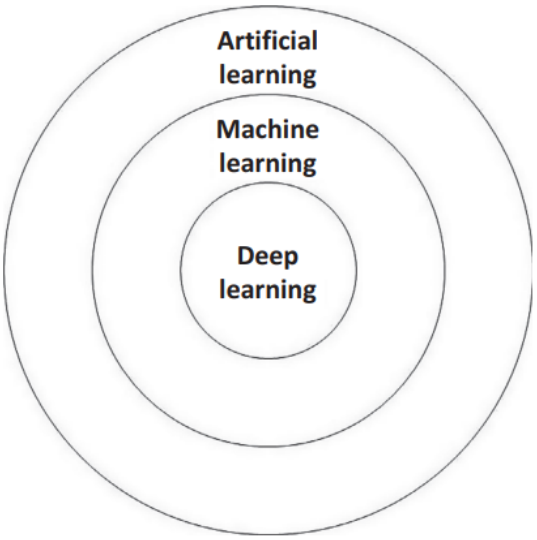
Além disso, é possível fazer o reconhecimento temporal (início, ápice e término) e a intensidade de uma determinada UA. Vale destacar que são as combinações específicas de UAs que tornam possível o reconhecimento de emoções. Ou seja, uma expressão emocional nada mais é do que um conjunto de UAs. (LIBRALON, 2014).

2.4 Inteligência Artificial

A inteligência artificial é um ramo da ciência da computação, que utiliza de algoritmos definidos por especialistas para desenvolver um sistema que simule a capacidade humana na percepção de um problema, na identificação de componentes e com base nestas análises seja capaz de tomar decisões, ou seja, é capaz de aprender e propor soluções.

Devido aos grandes avanços alcançados na área, as inteligências artificiais são capazes de identificar fala humana, competir em alto nível em jogos de estratégia, dirigir carros de forma autônoma, gerar simulações militares e interpretar conjuntos complexos de dados.

Figura 2 - Relação entre inteligência artificial, aprendizado de máquina e aprendizado profundo



Fonte: D. Jakhar, I. Kaur(2020).

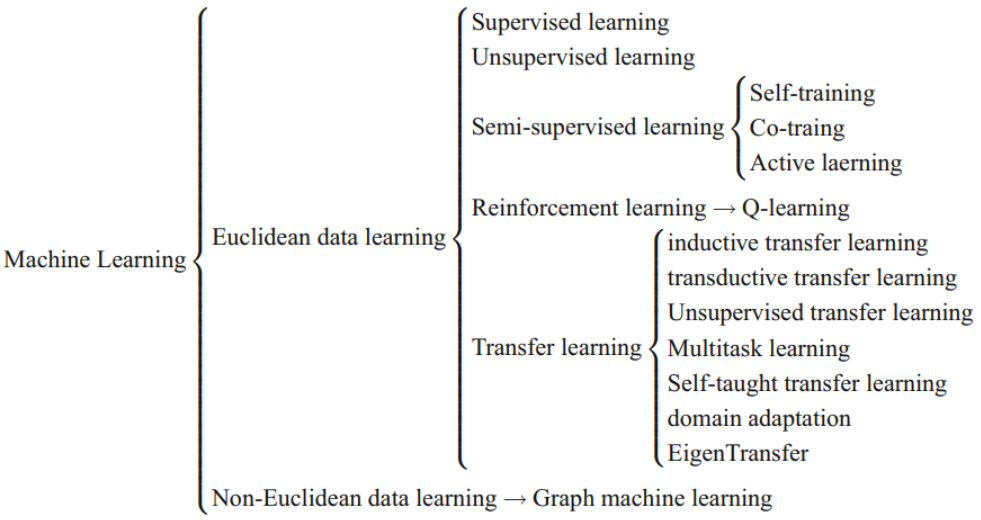
2.5 Aprendizado de Máquina

O aprendizado de máquina é uma subárea da Inteligência Artificial com o objetivo de desenvolver técnicas e modelos computacionais sobre o aprendizado, aplicado na construção de sistemas que buscam obtenção de conhecimento de forma automática. Baseando-se nestes modelos, dados e experiências passadas bem sucedidas, a máquina fará previsões e tomará decisões de forma espontânea, sem estar programada para tal.

O processo de aprendizado é iniciado a partir de um modelo matemático construído a partir de uma amostra de dados (massa de treinamento). A partir desta massa de treinamento, a máquina tem como objetivo estabelecer um regressor ou classificador através do aprendizado. O processo então é continuado com a introdução de novas massas de testes até então não apresentados ao sistema, se unindo com as massas anteriores já rotuladas pela máquina. A partir deste novo insumo de dados, é feita uma avaliação de performance do regressor ou classificador no aprendizado deste insumo de dados.

Conforme Xian-Da Zhang (2016), podemos classificar os tipos de aprendizado de máquina com base na imagem abaixo.

Figura 3 - Tipos de aprendizado de máquina.



Fonte: ZHANG (2020).

1. Regular ou Aprendizado de máquina Euclidiano estruturado
   1.1. Aprendizado supervisionado: a partir de uma massa de dados de treinamento já rotulados (com exemplos de entradas e as saídas esperadas), este tipo de aprendizado tenta aprender uma regra geral que mapeia as entradas às saídas.
   1.2. Aprendizado não supervisionado: Com um conjunto de dados não rotulados, espera-se que a máquina encontre as soluções de maneira espontânea (encontrando estruturas, padrões e modelos que se apliquem aos dados de entrada).
   1.3. Aprendizado semi-supervisionado: é fornecido à máquina uma pequena massa de dados rotulados e uma grande massa de dados não-rotulados. Este tipo de aprendizado se encontra entre o não-supervisionado e o supervisionado, e possui subcategorias dependendo de como os dados são rotulados.
      1.3.1. Autotreinamento: sistema de aprendizado semi-supervisionado, com base nas predições feitas pela máquina ocorre o aprendizado.

1.3.2. Co-treinamento: sistema de aprendizado semi-supervisionado, baseado em múltiplas visões do dado, e utiliza suas predições para aprenderem.

1.3.3. Aprendizagem ativa: sistema de aprendizado semi-supervisionado, onde a máquina possui papel ativo ou participatório no processo de determinação de quais data points serão rotulados por um especialista.

1.4. *Reinforcement learning*: Dados de treinamento são fornecidos como meios de recompensa ou punição, como um meio de feedback à inteligência artificial dentro de um ambiente dinâmico. Os feedbacks fornecidos ao sistema de aprendizado permitem uma melhora de performance no processo de aprendizado da máquina. Um exemplo seria o *Q-learning*.

1.5. Aprendizado por transferência: Se baseia na transferência do conhecimento de um conjunto de dados específicos de um domínio para outro domínio, dentro deste tipo de aprendizado temos os seguintes subtipos:

1.5.1. Aprendizagem por transferência indutiva: Neste subtipo os domínios de origem e destino são os mesmos, porém possuem tarefas diferentes. O algoritmo então busca utilizar o viés indutivo obtido pelo domínio de origem para melhorar a performance da tarefa do domínio destino.

1.5.2. Aprendizagem por transferência transdutiva: o processo de aprendizado deste subtipo dá-se através dos dados rotulados presentes no domínio de origem, inferindo rótulos aos dados do domínio destino. Este tipo de aprendizado ocorre quando existem similaridades nas tarefas de ambos os destinos, e quando a origem possui muitos dados rotulados, enquanto o destino não possui.

1.5.3. Aprendizagem por transferência não supervisionada: semelhante ao *inductive transfer learning*, porém possui foco nas tarefas não supervisionadas no domínio destino. Os destinos são semelhantes, porém as tarefas são diferentes, e neste cenário não há dados rotulados.

1.5.4. Aprendizagem multitarefa: é uma abordagem do *inductive transfer learning*, onde é melhorada a generalização com o uso das informações do domínio, obtidas das tarefas de treinamento, sendo aplicadas em outras tarefas. Este tipo de aprendizado executa diversas tarefas de treinamento, ao mesmo tempo, e aplica seus resultados em outras tarefas, auxiliando nas outras tarefas de aprendizado.

1.5.5. Aprendizagem por transferência autodidata: Este tipo de aprendizado começa com a entrada de um conjunto de dados não-rotulados, onde passam por um processo de aprendizado para obter uma melhor representação dos dados inseridos no sistema. Logo após, aplica o seu entendimento em um conjunto de dados rotulados, adquirindo um dado com maior grau descritivo do dado rotulado.

1.5.6. Adaptação de domínio: é a aplicação de um algoritmo treinado em diferentes domínios, em um diferente domínio destino.

1.5.7. *EigenTransfer*: Este tipo de aprendizado utiliza de grafos para representar o destino durante as tarefas deste tipo de aprendizado. Com o aprendizado do espectro do grafo é obtido um grupo de *eigenvectors*, representando a estrutura intrínseca do grafo da tarefa. A partir disso, os *eigenvectors* podem ser utilizados como o novo meio de transferência de conhecimento de dados auxiliares para classificar um dado alvo.

2. Aprendizado de máquina por grafos é um meio de aprendizado irregular ou não-euclidiano, e aprende a estrutura de um grafo a partir dos conjuntos de dados de treinamento em casos semi-supervisionados e não-supervisionados.

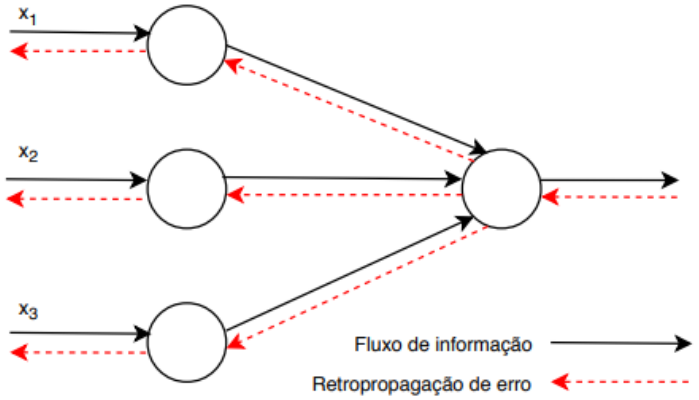
2.6 Aprendizado Profundo

Aprendizado Profundo é uma subárea de aprendizado de máquina, inspirada pela estrutura e funcionamento do cérebro, simulado por redes neurais artificiais. Ele permite o aprendizado de múltiplas representações do dado com múltiplos níveis de

abstração, a partir de modelos computacionais compostos por diversas camadas de processamento (SILVA, 2019).

Este método permitiu uma grande melhora no processo de reconhecimento de fala, reconhecimento de objetos visuais, detecção de objetos, e em muitos outros pontos, como o descobrimento de substâncias e na área de genômica. Este tipo de aprendizado de máquina permite a descoberta de estruturas complexas em grandes conjuntos de dados. Através do uso de algoritmos retro propagadores, a máquina recebe indicações de como ela deveria mudar seus parâmetros internos utilizados para computar a representação de cada camada, com base na camada anterior, assim obtendo representações cada vez mais complexas, e com maior grau de abstração (TRINDADE, 2018).

Figura 4 - Exemplo de retropropagação.



Fonte: TRINDADE (2018).

No aprendizado de máquina tradicional, parte dos problemas necessitam que sejam feitos um pré-processamento dos dados, e isso implica na necessidade de um conhecimento prévio. Um exemplo é o desenvolvimento de uma aplicação que faz o mapeamento de dados que não são lineares em uma representação linear, necessária para alguns algoritmos de aprendizado de máquina. Uma maneira de contornar essa necessidade é fazer com que o algoritmo aprenda não só a mapear os dados de entrada tratados para a saída, como também a representação necessária para viabilizar tal mapeamento. Outra dificuldade comum encontrada no aprendizado de máquina tradicional é quando o aprendizado das tarefas complexas

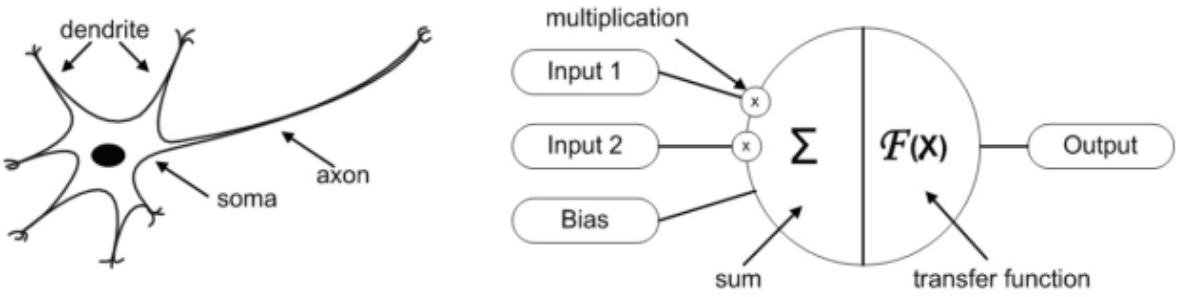
ou conceitos abstratos são tão complexos quanto o aprendizado da função em si. Nesse sentido, o aprendizado profundo é uma área do aprendizado de máquina responsável pela abstração dessas representações complexas, a partir de representações mais simples (BOCHIE, et al., 2020).

A quebra de um problema complexo em pequenos problemas possibilita que partes do algoritmo se especializem em tarefas simples, que depois são reagrupadas nas etapas seguintes, para permitir a resolução da tarefa mais complexa. No reconhecimento de imagem, por exemplo, um algoritmo pode ser dividido em módulos responsáveis por identificar partes da imagem, como as bordas. No decorrer da identificação, o módulo responsável pela identificação da borda pode ser combinado com um outro módulo responsável pela identificação da moldura, e assim sucessivamente, até que o conjunto dos módulos esteja especializado o suficiente para identificar a imagem. Com isso, o modelo de aprendizado é dividido em camadas de processamento. As camadas são organizações paralelas das estruturas de processamento básicas, que permitem a extração de informações progressivamente mais complexas, à medida em que os dados são processados através das múltiplas camadas intermediárias, de forma hierárquica. Apesar da definição de aprendizado profundo englobar quaisquer modelos de aprendizado que seguem esse modelo de processamento, o termo é mais comumente usado para se referir às Redes Neurais Profundas (BOCHIE, Kaylane, et al., 2020).

2.6.1 Redes Neurais

Inspirado no funcionamento do complexo cérebro humano, foi desenvolvido um modelo matemático que busca simular as estruturas e funcionalidades biológicas do cérebro humano. A construção de uma rede neural é feita a partir de um conjunto de neurônios artificiais. Um neurônio artificial é um bloco básico para a construção de uma rede neural, criado a partir da observação de neurônios biológicos presentes em estruturas como o cérebro e medula espinhal, e que são compostos pelos mesmos.

Figura 5 - Exemplo de um neurônio artificial.



Fonte: Artificial Neural Networks - Methodological Advances and Biomedical Applications (2011).

A partir desta observação chegou-se ao modelo apresentado acima, onde o neurônio artificial sumariza as entradas com peso e a predisposição, então realiza esta operação com base em uma função de ativação.

Figura 6 - Representação matemática de um neurônio artificial.

$$y(k) = F\left(\sum_{i=0}^{m} w_i(k) \cdot x_i(k) + b\right)$$

Fonte: Artificial Neural Networks - Methodological Advances and Biomedical Applications (2011).

Onde:

- $x_i(k)$ é o valor de entrada em tempo discreto k onde i vai de 0 a m
- $w_i(k)$ é o valor do peso em tempo discreto k onde i vai de 0 a m
- b é a predisposição
- F é a função de ativação
- $y_i(k)$ é o valor de saída em tempo discreto k

2.6.2 Funções de Ativação

Funções de ativação, função de limite ou função de transferência são funções matemáticas aplicadas no contexto de redes neurais artificiais, para transformar sinais de entrada em sinais de saída, dos quais serão utilizadas como entrada para as próximas camadas. Podemos considerá-la como a unidade mais importante de uma rede neural, por definir a saída de um neurônio dada uma entrada ou um conjunto de entradas (KARLIK, [2011?]).

Existe um grande número de funções de ativação, e há uma classificação entre elas com base em suas diferentes propriedades matemáticas, sendo não linear, alcance e continuamente diferenciável.

2.6.2.1 Sigmóide

É a função de ativação mais utilizada e não linear. A função Sigmóide transforma os valores de entrada em valores entre 0 e 1, pode ser representada da seguinte forma:

f(x) = $1/e^{-x}$

A função Sigmóide é continuamente diferenciável e uma função suave em forma de S, ela também não é simétrica em relação à zero.

2.6.2.2 Tanh

É uma função tangente hiperbólica, e possui semelhanças com a Sigmóide, mas ela é simétrica em torno da origem. Isso resulta em diferentes sinais de saídas de camadas anteriores, que serão alimentadas como entrada para a próxima camada. Pode ser definida como:

$$f(x) = 2sigmoid(2x) - 1$$

A função Tanh é contínua e diferenciável, os valores variam entre -1 e 1. Esta

função é preferível à função Sigmóide, por possuir gradientes que não estão restritos a variar em uma determinada direção e, também, por estar centrada em zero.

2.6.2.3 ReLU

ReLU faz referência a unidade linear retificada e é uma função não linear, com vasto uso em redes neurais. Pode ser definida como:

$$f(x) = max(0, x)$$

ReLU é mais eficiente que outras funções por não ativar todos os neurônios no mesmo momento.

2.6.2.4 SoftMax

A função *SoftMax* é uma combinação de múltiplas funções sigmóides. A função Sigmóide retorna valores entre 0 e 1, que podem ser tratadas como probabilidade de determinada classe.

Diferente da função Sigmóide que é utilizada para classificação binária, a *SoftMax* pode ser utilizada para classificação de múltiplas classes. A função retorna a probabilidade de cada classe para cada dado. A função pode ser expressa como:

$$\sigma(\mathbf{z})_j = \frac{e^{z_j}}{\sum_{k=1}^{K} e^{z_k}}$$ Onde j = 1, …, K.

Quando construímos uma rede neural ou modelo com classificação de múltiplas classes, o número de classes alvos definirá o número de neurônios presentes na camada de saída.

2.6.3 Otimizador

Uma rede neural exige que as variáveis de cada camada sejam alteradas, com o objetivo de fazer o melhor processo de classificação possível. Para essa tarefa, é preciso que o resultado obtido seja sempre comparado com o valor esperado. O objetivo do otimizador é minimizar o resultado da função *cross-entropy* (entropia cruzada), que sempre é positivo e vira zero quando o valor obtido é igual ao esperado, e auxilia na criação das redes neurais. Existem alguns métodos de otimização que trabalham de forma diferente e cada uma deve ser usada em diferentes casos, para que se possa chegar a um melhor resultado (RIBEIRO; JUNIOR, 2020).

2.6.3.1 Adam

O método *Adam* faz o cálculo da taxa de aprendizado adaptativa para cada parâmetro. Essa metodologia mantém uma média exponencial decrescente dos gradientes anteriores m_t, sendo $m_t = \beta_1 m_t - 1 + (1 - \beta_1) gt$ e $v_t = b_2 v_t - 1 + (1 - \beta_2) g_t^2$, onde m_t e v_t são estimativas do momento inicial, a média, e o segundo momento, a variação não centralizada, dos gradientes respectivamentes, β_1 e β_2 são as taxas de decaimento e elas terminam em 1. A regra de atualização se dá por: $\theta_{t+1} = \theta_t - \frac{n}{\sqrt{\widehat{\upsilon_t} + \epsilon}} \widehat{m_t}$ (RIBEIRO; JUNIOR, 2020).

2.6.3.2 SGD

O método SGD funciona executando uma atualização de parâmetros para cada amostra de treino $x^{(i)}$ e para cada rótulo $y^{(i)}$: $\theta = \theta - \eta \cdot \forall_\theta J(\theta; x^{(i)}, y^{(i)}$. No SGD a atualização de parâmetro é executada uma de cada vez. Isso faz com que ele seja um algoritmo muito rápido, podendo ser usado em aprendizado online. O algoritmo executa atualizações com uma alta variância, isso faz com que função

objetivo sofra grandes flutuações. *Momentum* é um método que auxilia na aceleração do método SGD, fazendo-o encontrar o caminho correto de convergência com mais velocidade e apresentando menos oscilações. Isso ocorre devido a adição de uma fração do vetor de atualização do passo anterior ao atual vetor de atualização (RIBEIRO; JUNIOR, 2020).

2.6.3.3 RMSProp

O método RMSProp trabalha com uma taxa de aprendizagem adaptativa que combina a ideia de utilizar somente o sinal do gradiente com a ideia de adaptar de forma separada o tamanho de passo para cada peso. A média de execução $E[g^2]t$ num tamanho de passo t depende da média anterior e do gradiente atual onde $E[g^2]t = 0.9E[g^2]_{t-1} + 0.1g_t^2$ e $\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} \cdot g_t$. Esse algoritmo faz a divisão entre a taxa de aprendizado pela média exponencial decrescente do quadrado dos gradientes (RIBEIRO; JUNIOR, 2020).

2.6.3.4 Adagrad

O método *Adagrad* é um algoritmo de otimização que utiliza dinamicamente de conhecimento sobre os dados observados nas interações que foram feitas anteriormente a fim de executar um aprendizado que permite ao algoritmo reconhecer características raras mas predizíveis sobre os dados trabalhados. O método se baseia na descida do gradiente que adapta a taxa de aprendizado η para os parâmetros, executando grandes atualizações para parâmetros que não são frequentes e atualizações menores para os parâmetros mais frequentes. *Adagrad* altera a taxa de aprendizado η a cada intervalo de tempo t para cada parâmetro θ_i se baseando nos gradientes anteriores calculados para este parâmetro de acordo com $\theta_{t+1,i} = \theta_{t,i} - \frac{\eta}{\sqrt{G_{t,ii} + \epsilon}} \cdot g_{t,i}$, onde G_t é uma matriz diagonal onde cada elemento i, i é a soma dos quadrados dos gradientes com respeito a θ_i no intervalo t,

enquanto ϵ é um termo de suavização que evita divisão por zero e $g_{t,i} = \forall_{\theta} J(\theta_i)$, sendo $g_{t,i}$ o gradiente da função objetivo com respeito ao parâmetro θ_i no intervalo i. (RIBEIRO; JUNIOR, 2020).

2.6.4 Batch Size

O *batch size* (Tamanho do lote) refere-se a quantidade de exemplos de treinamentos que serão usados durante uma iteração. Ele pode ser de três tipos:

- *batch mode*: Nesse caso o tamanho do lote é igual ao conjunto de dados, dessa forma, a quantidade de iterações e épocas equivalentes.
- *mini-batch mode*: O tamanho do lote é menor que o conjunto de dados e maior que um.
- *stochastic model*: Nesse caso o tamanho do lote é igual a um. Fazendo com que o gradiente e os parâmetros da rede neural sejam atualizados após cada amostra.

2.6.5 TensorFlow

O *TensorFlow* é um *framework open-source* desenvolvido para Python, *Javascript*, C/C++ e *Go*. Foi criado pela equipe do *Google Brain* para aprendizado de máquina, para facilitar o processamento de dados através de um modelo de fluxo de dados. Seu uso vem aumentando pelo fato de seu desempenho usar do XLA, um compilador poderoso de álgebra linear que torna a execução mais acelerada, podendo ser executado em CPUs, GPUs, TPUs, entre outros.

O maior benefício da utilização do *TensorFlow* é a abstração que ele permite. Ao invés do desenvolvedor precisar se preocupar com todos os detalhes referentes a implementação do algoritmo, pode-se focar apenas na lógica geral da aplicação. Além de permitir a abstração de parte do desenvolvimento, o *TensorFlow* também permite que o usuário utilize algumas APIs para facilitar ainda mais o desenvolvimento (RIBEIRO; GUIMARÃES, 2018).

2.7 Detecção Facial

A primeira coisa a ser feita quando queremos fazer um reconhecimento de expressão facial é detectar o rosto na imagem que está sendo analisada. O objetivo é encontrar e delimitar o rosto nessa imagem da melhor forma possível, pois um reconhecimento impreciso pode levar a um mal desempenho do algoritmo que fará o reconhecimento de expressões posteriormente. É importante ressaltar que, caso um vídeo seja analisado, também será levado em conta a informação temporal (ROSEIRO; COSTA, 2020).

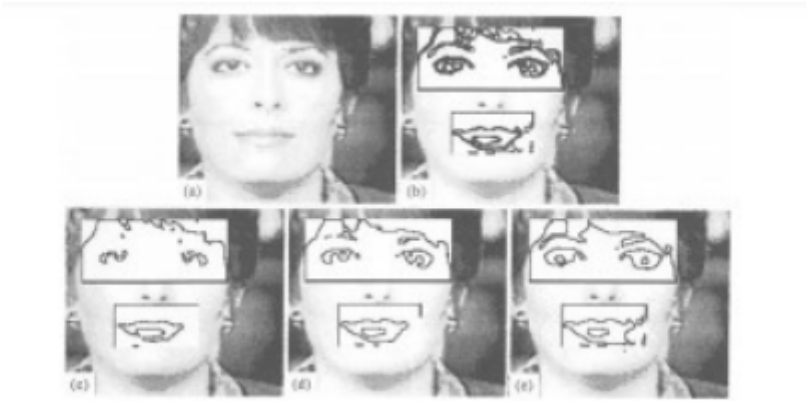
Muitos dos algoritmos que fazem o reconhecimento facial pressupõem que as condições em que a imagem é obtida são controladas. Normalmente, o rosto se encontra em posição frontal na sequência de imagens, nesses casos a presença da face é garantida na imagem e a localização tende a ser mais fácil. Porém, em alguns casos (e.g. imagens com ruídos, oclusão, posicionamento da face) a localização exata pode ser um pouco mais complexa (PATIL; SAHULA, 2010).

O ser humano consegue detectar rostos sem muito esforço, iluminação ou mesmo a longas distâncias, inclusive podendo detectar mesmo que parte do rosto esteja escondido, uma vez que o sistema perceptivo consegue completar a parte que fica faltando. Entretanto, fazer isso com uma máquina não é tão trivial. O sistema tem que lidar com movimentos da cabeça, deve conseguir identificar o rosto tanto na posição frontal quanto na de perfil, e também capaz de conseguir funcionar em ambientes com baixa iluminação e com distrações como barba, bigode e óculos (PATIL; SAHULA, 2010).

Alguns dos algoritmos de detecção facial precisam ser muito bem treinados para que possam obter bons resultados. Quanto mais eles são treinados, mais se espera que os resultados sejam positivos. No treinamento, temos que usar tanto imagens que tenham faces, quanto imagens sem faces. Com isso, podemos tratar o problema de detecção facial em duas categorias, onde uma corresponde às faces e a outra ao que não é face. A principal dificuldade para detectar a face em uma imagem é causado por não saber previamente em qual posição da imagem a

mesma se encontra, além disso, outro problema é originado por objetos que se assemelham a faces em imagens de baixa resolução (BRAGA, 2013).

Figura 7 - Exemplo de método de detecção facial.



Fonte: Silva (2018).

Existem duas formas importantes de avaliar a qualidade do algoritmo: o número de vezes que foi incorretamente identificado como face (falso positivo), e o número de vezes em que as faces não foram identificadas (falso negativo). Quanto mais próximo de zero, melhor é o resultado.

Os métodos de detecção facial são separados em quatro modos: métodos baseados em conhecimento, métodos baseados em características invariantes, métodos baseados em comparação de padrões e métodos baseados em aparência (DINIZ, 2013).

Os métodos baseados em conhecimento utilizam como base regras que são definidas a partir do conhecimento prévio do problema. Por exemplo, podemos dizer que grande parte das faces humanas possuem determinados atributos, como: na teoria todas as faces possuem dois olhos, boca e nariz. Além disso, existe uma ordem específica em que esses atributos estão posicionados sobre a face. Baseado nisso, é possível definir regras que reconhecem uma face humana. Existem algumas desvantagens específicas na utilização desse método. Se for usado regras muito genéricas, é possível que o algoritmo tenha um alto índice de falsos positivos (BISSI, 2013).

Os métodos baseados em características invariantes tem como objetivo encontrar características que são invariantes em todas as faces. Esses métodos

tentam identificar a face da mesma maneira que os seres humanos, ou seja, tem a capacidade de identificar de qualquer ponto de vista. Uma desvantagem desses métodos é que alguns fatores relacionados a imagem, como a iluminação ou algum tipo de ruído, podem afetar a eficiência de tal abordagem. As principais características invariantes que são utilizadas para distinguir a face de outros objetos são a cor da pele e a textura da face (BISSI, 2013).

Os métodos baseados em templates tem como objetivo procurar na imagem um objeto que corresponda a um modelo prévio. Uma das formas mais comuns de usar esses métodos é tentar descrever os objetos através de formas geométricas. Esta técnica é denominada template (BISSI, 2013).

Os métodos baseados em aparência não tem, a princípio nenhum, conhecimento sobre o objeto que vai ser detectado. Nesta classe de algoritmo são utilizados os conceitos de aprendizado de máquina, já que as informações necessárias para a detecção são coletadas do conjunto de imagem sem nenhuma intervenção externa (BISSI, 2013).

2.7.1 Algoritmo Viola-Jones

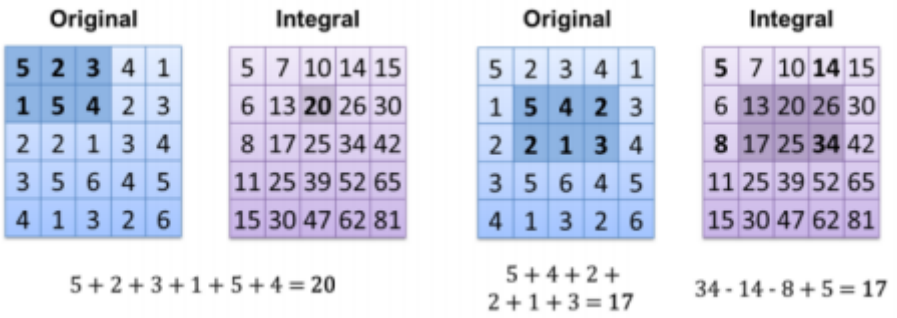
Idealizado por Viola e Jones em 2001, o algoritmo Viola-Jones é baseado em aparência e bastante utilizado pela comunidade por sua eficiência e facilidade na detecção de faces (SILVA, 2018). Ele é definido pelo OpenCV como um “Classificador *Haar*”, pois consiste em fazer cálculos de soma e subtração com regiões retangulares da imagem antes de chegar ao resultado final (PAIXÃO, 2018). Foi o primeiro algoritmo a ser capaz de detectar faces em tempo real em vídeo, conseguindo processar 15 quadros por segundo (OMAIA, 2009).

O algoritmo já está implementado em algumas bibliotecas como a OpenCV e a na OpenBR. Além de detectar faces, também é possível detectar outros objetos após ser feito o treinamento de detecção. O processo de treinamento costuma ser lento, porém o reconhecimento é rápido (SILVA, 2018).

O objetivo dessa técnica de reconhecimento de padrões é encontrar partículas de imagem que contenham diferenças de contraste que são comuns em imagens que possuem rostos, essa técnica é conhecida como *Haar*. Normalmente, para fazer a extração a imagem é convertida em escalas de cinza (PAIXÃO, 2018).

O método cria uma imagem integral para fazer a análise e localizar atributos retangulares o mais rápido possível. A imagem integral é uma matriz do mesmo tamanho da imagem que será analisada, sendo que o pixel atual é igual a soma da escala de cinza dos pixels que estão acima e a esquerda do mesmo. É possível verificar a criação da matriz na Figura 3 (OMAIA, 2009).

Figura 8 - Exemplo da geração da matriz de imagem integral.

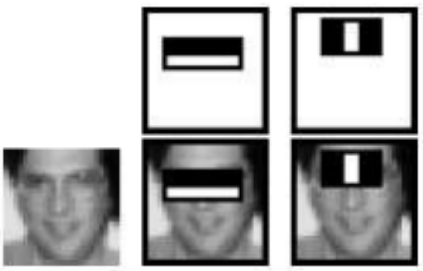


Fonte: Computer Vision Talks (2019).

As duas primeiras matrizes representam a criação da imagem integral, sendo que na primeira temos a imagem original e na segunda os valores da imagem integral. A terceira matriz demonstra como que o algoritmo seleciona um subconjunto da imagem original, com o objetivo de ter a soma de luminosidade dos pixels que estão na mesma. A quarta mostra como é feito o cálculo acessando apenas quatro posições de memória da matriz.

Depois da criação da imagem integral é feita a comparação de subconjuntos da mesma utilizando o *Haar-Like*, com o objetivo de saber se o padrão de luminosidade desses conjuntos são semelhantes aos que são encontrados em faces (ROSEIRO; COSTA, 2020).

Figura 9 - Comparação com a imagem original usando Haar-Like.

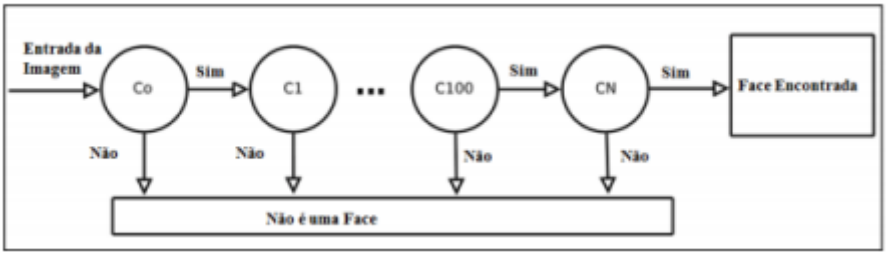


Fonte: (OMAIA, 2009).

Apesar do cálculo de cada subconjunto ser rápido, já que são feitos apenas quatro acessos à memória e uma soma com os valores encontrados, comparar todos os subconjuntos é um processo trabalhoso e inviável. Com o tempo foi descoberto que um classificador eficiente pode ser formado com a combinação de um pequeno subconjunto que contenha as características mais representativas. O algoritmo de Viola-Jones utiliza um derivado do *AdaBoost*, para fazer a seleção dessas características e treinar o classificador (SILVA, 2018).

O *AdaBoost* é um algoritmo de aprendizado de máquina que foi criado por Yoav Freund e Robert Schapire, que faz a combinação entre vários classificadores fracos, para obter um classificador forte (SILVA, 2018). O método Viola-Jones faz a combinação em cadeias de um conjunto de classificadores *AdaBoost* em cadeia, com isso a análise se torna mais rápida, pois caso a resposta de um dos classificadores seja negativa, a imagem já é classificada automaticamente como não sendo face. Caso o classificador passe por todas as etapas e o resultado seja favorável, a imagem é classificada como contendo face (DINIZ, 2013).

Figura 10 - Exemplo de classificadores em cadeia.



Fonte: Diniz (2013).

2.7.2 MTCNN

O MTCNN (*Multi-task Cascaded Convolutional Networks*, ou Redes convolucionais em cascata multitarefa) é um algoritmo que foi desenvolvido por ZHANG em 2016, que utiliza três redes neurais para detectar uma região de interesse (CANTO, 2019). O método tem como entrada imagens, e como saída

todas as faces que foram encontradas na mesma, sendo que cada uma delas possui algumas informações, como (ROSEIRO; COSTA, 2020):

- As coordenadas do retângulo em que se encontra a face.
- Cinco pontos relevantes no rosto, como: olhos, nariz e extremidades da boca.
- O nível de confiabilidade da detecção.

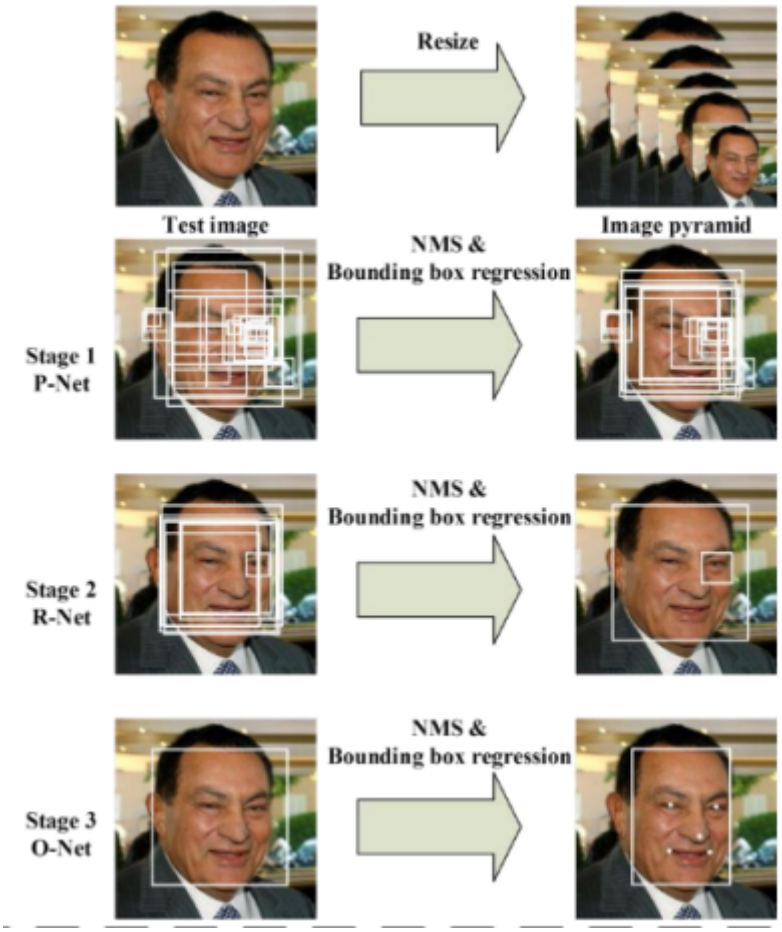
Após o treinamento do algoritmo, o mesmo redimensiona a imagem em diversas escalas para a utilização de um conceito chamado *sliding windows*. Esse conceito serve para varrer as imagens em busca de classificações que indiquem presença de uma face (CANTO, 2019). Depois do redimensionamento, o processo que ocorre é dividido em três níveis de redes neurais convolucionais, ou CNNs, todas elas unificadas em um processo de aprendizado multitarefa proposta por Caruana, sendo que o resultado do primeiro nível é compartilhado com o seguinte e assim por diante, com o objetivo de melhorar a performance do algoritmo (ROSEIRO; COSTA, 2020).

O primeiro nível é chamado de P-NET e tem o objetivo de prever a localização da face e as suas delimitações (CANTO, 2019). A imagem é enviada para a rede neural para que seja possível a coleta dos vetores que representam as extremidades do objeto que foi encontrado, e também é utilizado um método de supressão que elimina os candidatos com pontos comuns (FAJARDO, 2019).

No segundo nível, algumas imagens são classificadas para ser feita uma nova filtragem. A arquitetura utilizada para isso é a R-NET, com o intuito de remover os candidatos que não possuem face (CANTO, 2019).

No terceiro nível é feita uma filtragem ainda maior com os resultados e marcações faciais. Nesse nível é usado a arquitetura O-NET, com o intuito de marcar as cinco posições faciais mais importantes: os dois olhos, o nariz e as extremidades horizontais da boca (CANTO, 2019).

Figura 11 - Funcionamento MTCNN.



Fonte: Zhang (2016).

2.7.3 Lucas-Kanade

Lucas-Kanade é um método diferencial, ou seja, ele utiliza equações diferenciais para fazer o cálculo do fluxo óptico que foi criado por Bruce Lucas e Takeo Kanade. O algoritmo supõe que em determinada região de pixels o fluxo é constante. Dessa forma, é utilizado o método de mínimos quadrados para resolver as equações para os pixels que se encontram naquela região (LEÃO, BEZERRA, MATOS e NUNES, 2012).

Como o método assume que o fluxo óptico entre os pixels que são vizinhos é constante em pequenas janelas $m \times m$, onde $m > 1$, para todos os pixels ordenados de 1 a n, o conjunto de equações pode ser determinado da seguinte maneira:

Figura 12 - Conjunto de equações.

$$
\begin{aligned}
I_{x1}V_x + I_{y1}V_y &= -I_{t1} \\
I_{x2}V_x + I_{y2}V_y &= -I_{t2} \\
&\vdots \\
I_{xn}V_x + I_{yn}V_y &= -I_{tn}
\end{aligned}
$$

Fonte: ISHII (2018).

A equação pode ser resolvida utilizando o método dos mínimos quadrados e, além disso, a equação pode ser escrita de forma matricial, como por exemplo:

Figura 13 - Equação na forma matricial.

$$A \times v = B$$

$$A = \begin{bmatrix} I_{x1} & I_{y1} \\ I_{x2} & I_{y2} \\ \vdots & \\ I_{xn} & I_{yn} \end{bmatrix},$$

$$v = \begin{bmatrix} V_x = \frac{dx}{dt} \\ V_y = \frac{dy}{dt} \end{bmatrix},$$

$$B = \begin{bmatrix} -I_{t1} \\ -I_{t2} \\ \vdots \\ -I_{tn} \end{bmatrix}$$

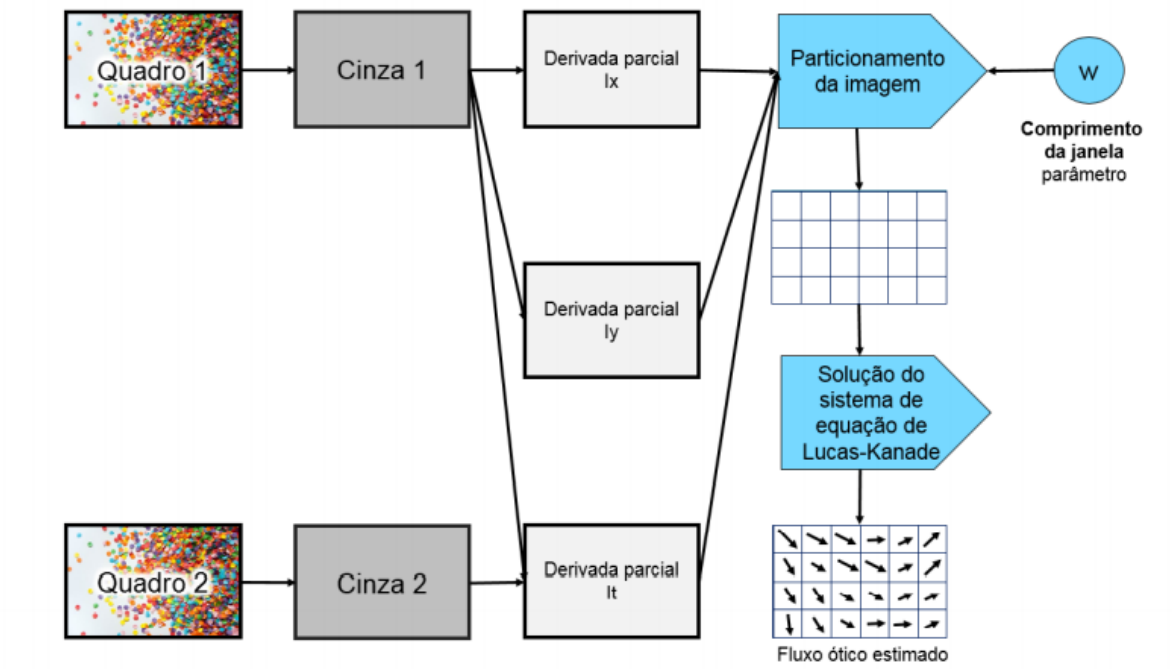
Fonte: ISHII (2018).

Na Figura 8 ilustra como o processo de estimativa de fluxo óptico funciona. É importante lembrar que a imagem precisa ser convertida para escala de cinza antes que ela possa ser usada como entrada para o método Lucas-Kanade.

Após isso, são feitos os cálculos das derivadas parciais da imagem, para que seja medida a diferença entre o tom de cinza dos pixels vizinhos. Na sequência a imagem é particionada em regiões na qual o fluxo óptico é constante, ou seja, com isso é possível fazer o mapeamento das novas posições em que os elementos da face se encontram (ISHII, 2018).

O motivo da escolha do algoritmo se deu por ser possível fazer o remapeamento constante das posições dos elementos faciais. Um exemplo é que, dessa forma, é possível identificar a mudança de posição da boca em cada frame de um vídeo.

Figura 14 - Técnica de estimativa de fluxo-óptico.



Fonte: ISHII (2018).

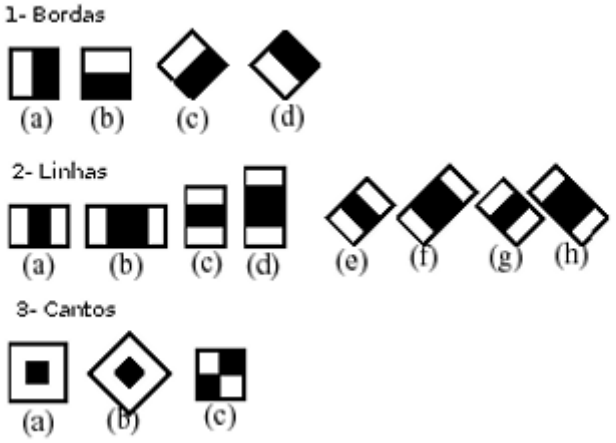
2.7.4 Haar-Cascade

O *Haar Cascade* é um algoritmo de aprendizado de máquina que permite uma grande liberdade em relação ao alvo a ser detectado, pois possui algoritmos rápidos

e de baixo custo computacional para fazer a detecção em tempo real. Além disso, é dito como uma forma robusta, por ser capaz de fazer um treinamento com objetos que estão em condições adversas como baixa luminosidade. O algoritmo se baseia em quatro conceitos fundamentais para conseguir realizar a detecção de objetos: características de *Haar*, imagens integrais, *Adaboost* e os classificadores em cascata (MACEDO; GOMES, 2020).

As características de *Haar* consistem na verificação da diferença da intensidade entre regiões retangulares da imagem. Cada uma dessas verificações resulta em um valor que é obtido após subtração da soma dos pixels retangulares brancos pela soma dos pixels retangulares pretos. A imagem a seguir mostra as características da borda, linha e cantos de uma imagem.

Figura 15 - Características de Haar.

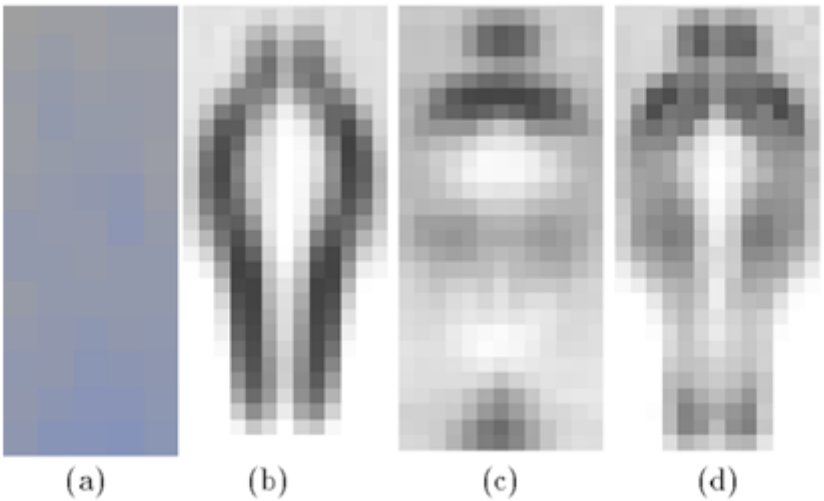


Fonte: SHIMABUKURO e SOUSA (2011).

As vantagens de seu uso podem ser descritas pela sua eficiência computacional, a independência de escala e cor, baixa variedade intra-classes, a alta variação inter-classes e o fato de ser orientada a diferença de intensidades locais. A diferença da intensidade nas regiões da imagem pode ser computada facilmente com características *Haar*, como podemos ver na figura 14. Quando foi aplicado uma função que calcula a diferença entre essas regiões, podemos verificar

certos padrões de mudança nas sub-regiões da imagem indicando uma mudança drástica, em casos de borda, ou que a região é uniforme.

Figura 16 - Média de intensidade em tons de cinza.



Fonte: SHIMABUKURO e SOUSA (2011).

Esses padrões podem ser observados na figura 15, na qual (a) representa os coeficientes verticais de uma imagem negativa e (b), (c) e (d) representam os coeficientes obtidos da aplicação de características *Haar* vertical, horizontal e de cantos em uma imagem positiva dimensionada para 16 x 16.

Pelo custo computacional de analisar uma imagem utilizando as características de *Haar* em diferentes escalas, Paul Viola e Michael Jones (2001) criaram uma representação intermediária da imagem, chamada de Imagem Integral, com isso a computação se tornou possível com poucas operações por pixel. Após o cálculo, pode ser obtida em qualquer escala em tempo constante.

O cálculo da imagem integral é feito de maneira simples. Considera-se um ponto (x, y) em determinada imagem, relaciona-se com o valor na imagem integral os pixels que estão acima e a esquerda do ponto (x, y) escolhido. Dessa forma, os valores são calculados fazendo uma varredura na imagem original, o valor de intensidade de um pixel na imagem integral I(x,y) sendo: I(x,y) = o(x,y) + I(x-1,y) + I(x,y-1)- I(x-1,y-1), onde o(x,y) é o pixel na imagem original (SOUSA; SHIMABUKURO, 2011).

Após a obtenção das características, é preciso um algoritmo de aprendizado que consiga fazer o reconhecimento das características de *Haar* na imagem. O algoritmo *Adaboost*, que possui um conjunto de classificadores pré-treinados e um conjunto de avaliação, retro-alimenta os pesos dos classificadores existentes, atualizando a probabilidade de reconhecimento do evento. O algoritmo faz isso combinando regras simples de treinamento.

Visto que durante o processamento de uma imagem pode-se ter mais características de *Haar* do que pixels em determinada região, foi proposto por [Paul Viola e Michael Jones 2001] um modelo de classificadores em cascata, no qual classificadores mais simples fazem avaliações de sub-janelas da imagem, excluindo aquelas são explicitamente negativas. Ou seja, não corresponde ao que está sendo detectado. Após isso, os classificadores mais robustos e mais complexos, avaliam a presença do alvo somente nas sub-janelas que não foram negativadas na fase anterior. E isso é feito sucessivamente, de forma que os classificadores mais robustos ficam responsáveis por um processamento mais simples, reduzindo a exigência de poder computacional.

2.8 Análise Técnica

Segundo Debastiani (2008), a análise técnica é um conjunto de métodos e ferramentas aliadas à observação de comportamentos passados, para poder identificar futuras tendências.

Dentro de análises técnicas, temos duas principais: a análise técnica empírica é baseada na observação de acontecimentos passados, tornando possível identificar certos comportamentos e tendências a partir de dados passados, e torna possível a identificação de ciclos repetitivos, possui certo grau de confiabilidade pela sua presença constante e repetitiva, e a análise técnica estatística é baseada em modelos matemáticos e estatísticos, montadas a partir de dados passados, possui maior assertividade e confiança, devido aos modelos.

Este tipo de análise possui grande utilidade para avaliar a performance do reconhecimento e classificação de emoções, nos possibilitando ter noção do quão

eficaz ele está sendo e comparar com a performance de outros trabalhos relacionados.

2.8.1 Acurácia

A acurácia serve para definir a proximidade de um resultado experimental com o valor real. Significa que quanto maior a acurácia, mais próximo do valor real ele é. Na tecnologia, acurácia é utilizada como proximidade de um resultado experimental com o valor de referência real. Assim, ela determina o grau de exatidão, ou seja, é uma forma de medir o nível de erro entre o valor ideal e o valor real.

Existe uma diferença entre acurácia e precisão. A precisão é o grau de variação gerado por diferentes medições. Quanto mais preciso for um processo, menor será a variação entre os valores obtidos. Já a acurácia é a soma entre a exatidão e precisão, dessa forma, os resultados obtidos por uma experiência precisam estar próximos do valor de referência ou real usado como base, além de serem precisos.

2.9 Trabalhos relacionados

2.9.1 Recognition of emotions from video using acoustic and facial features

Neste artigo, é buscado realizar o reconhecimento de emoções através de um vídeo, extraindo dele a acústica e as expressões faciais. Para captar as variações conforme o tempo, é analisada as variações dos pixels nos olhos e na região da boca, através de uma rede neural auto associativa, que também captura a acústica. Com a captura destes dados, é feito um reconhecimento dos sinais faciais e acústicos, que estão relacionados a algum tipo de emoção.

Tem como finalidade capturar e analisar a capacidade de detecção de emoções através da acústica e das expressões faciais, aplicando diferentes modelos (RAO e KOOLAGUDI, 2013).

2.9.2 RedFace – Um Sistema de Reconhecimento de Expressões Faciais para Apoiar um Ambiente Virtual de Aprendizagem.

O objetivo deste trabalho é desenvolver um Ambiente Virtual de Aprendizagem que seja capaz de fazer a autenticação do aluno pelo reconhecimento facial e melhorar a qualidade do ensino baseado nas análises de expressões faciais feitas durante o uso do sistema.

Foi utilizado o Viola-Jones para fazer a detecção facial na imagem e, em seguida, aprendizado de máquina para fazer a classificação de emoção das faces analisadas (DINIZ, 2013).

Figura 17 - Performance da análise de reconhecimento facial e acústico combinados.

Table 7 Performance of the combined emotion recognition system by combining the evidences of individual ER systems developed using acoustic and facial features

Emotion	Emotion recognition performance (%)		
	Acoustic (Phase-3)	Facial (Phase-1(b))	Acoustic + Facial (Phase-4)
Anger	87.81	86.72	95.78
Fear	83.47	87.91	91.14
Happy	80.32	92.78	93.26
Neutral	91.65	87.89	94.09
Sad	85.28	85.42	93.88

Average emotion recognition performance of the combined (acoustic+facial) emotion recognition system is 93.62 %

Fonte: (RAO e KOOLAGUDI., 2013).

2.9.3 Detecção de expressões faciais: Uma abordagem baseada em análise do fluxo óptico.

Neste trabalho, o objetivo é construir um sistema que consiga fazer o reconhecimento das expressões faciais em tempo real utilizando o algoritmo *Haartraining*, posteriormente fazendo o mapeando do rosto e determinando pontos-chave para fazer a análise dos movimentos, utilizando a técnica de fluxo óptico. Em seguida, é feita a interpretação das unidades de ação propostas por Paul Ekman.

Por fim, é possível detectar automaticamente as seis emoções básicas (raiva, medo, felicidade, tristeza, aversão e surpresa) (LEÃO, BEZERRA, MATOS, NUNES, 2012).

Figura 18 - Precisão do trabalho.

Tabela 2. Matriz de confusão para as seis expressões faciais: felicidade (F), medo (M), surpresa (S), tristeza (T), raiva (R) e aversão (A). As linhas representam a expressão intencionada pelo usuário, já as colunas representam a expressão detectada pelo sistema. A precisão média foi de 68,89%

	F	M	S	T	R	A
F	73,3%	6,7%	0%	0%	0%	20%
M	0%	80%	6,7%	0%	0%	13%
S	0%	40%	60%	0%	0%	0%
T	0%	0%	0%	67%	0%	33%
R	6,7%	0%	0%	0%	53,3%	40%
A	0%	20%	0%	0%	0%	80%

Fonte: (LEÃO, BEZERRA, MATOS, NUNES, 2012).

3 Desenvolvimento

3.1 Solução proposta

O sistema consiste na utilização do *TensorFlow* para reconhecimento facial e classificação de emoção das imagens analisadas. O sistema foi dividido em três partes, o *front-end, back-end* e a inteligência artificial.

No *front-end* foi utilizado o Angular na versão 12, por ser a mais recente no momento em que foi iniciado o desenvolvimento. Em nossa página, foi embutido um vídeo do *YouTube* e solicitado ao usuário permissão para ativar a câmera do dispositivo. Com isso, ocorre a gravação da reação dele enquanto assiste ao vídeo do *YouTube*. A cada 3 segundos é capturado um frame da gravação do usuário, fazendo a detecção e mapeamento do rosto usando o *TensorFlow*. Caso o rosto seja detectado, é enviado para o *back-end* via API.

No *back-end* foi utilizado o Python 3 e o MongoDB como banco de dados. O Python foi escolhido por ser uma linguagem que permite mais facilidade para lidar com dados e na construção de uma API. O MongoDB foi escolhido pela não necessidade de um banco relacional e, além disso, o MongoDB possibilita uma maior flexibilidade na mudança das colunas da tabela. Foi criada uma API que recebe a imagem com as coordenadas do rosto mapeado, em formato Base64, e salva no banco de dados, para posteriormente realizar a classificação de emoção e devolver para o *front-end*.

3.2 Detecção Facial

Utilizamos o *TensorFlow* por ser a biblioteca que melhor se adaptou às condições, utilizando-se o *TensorFlow MediaPipeFaceDetector* no *front-end* para a detecção e mapeamento facial.

O *MediaPipeFaceDetector* é uma solução de detecção de face ultra rápida, que disponibiliza seis pontos de referência e reconhece mais de uma face em uma imagem. Baseado no *BlazeFace*, um detector de face leve e de bom desempenho, permite o reconhecimento em tempo real, de forma que possa ser usado em

qualquer contexto, em que seja necessário fazer o reconhecimento facial em um vídeo ao vivo que exija uma região facial precisa para a entrada em outros modelos.

Como pode ser visto na figura a seguir, primeiro foi iniciada a variável *model* com o *BlazeFace* do *TensorFlow*, para na sequência passarmos o frame do vídeo que está sendo capturado pela webcam como parâmetro no método do *BlazeFace*.

Figura 19 - Código de detecção facial.

```
async ngOnInit(): Promise<void> {
  this.model = await blazeface.load();
  window.addEventListener('DOMContentLoaded', this.openCamera);
}


async takeSnapshot(videoTime: Number) {
  //Captura elemento de vídeo
  const video = document.querySelector<HTMLVideoElement>("#webcam-video");

  if(video != null){
    //Criando um canvas que vai guardar a imagem temporariamente
    var canvas = document.createElement('canvas');
    canvas.width = video.videoWidth;
    canvas.height = video.videoHeight;

    this.predictions = await this.model.estimateFaces(video, false);
```

Fonte: Autoria própria.

Caso a face seja detectada na imagem que foi passada por parâmetro, a variável *predictions* recebe as coordenadas com o mapeamento do rosto e com os pontos-chave, como a posição dos olhos, nariz, orelhas, entre outros. Podemos ver um exemplo na imagem a seguir.

Figura 20 - Resultado da detecção facial.

```
[
  {
    box: {
      xMin: 304.6476503248806,
      xMax: 502.5079975897382,
      yMin: 102.16298762367356,
      yMax: 349.035215984403,
      width: 197.86034726485758,
      height: 246.87222836072945
    },
    keypoints: [
      {x: 446.544237446397, y: 256.8054528661723, name: "rightEye"},
      {x: 406.53152857172876, y: 255.8, "leftEye },
      ...
    ],
  }
]
```

Fonte: Autoria Própria.

O código fonte do módulo *Blazeface* pode ser encontrado no repositório do *TensorFlow*.

3.3 Massa de dados

A escolha do conjunto de dados é de extrema importância no processo de treinamento de uma rede neural. Os dados de entrada impactam diretamente nos resultados da rede neural e, também, no progresso de aprendizagem. Durante o processo de seleção do conjunto de dados que será escolhido, é importante considerar o volume de dados presentes no conjunto, a qualidade dos dados e a forma em que os dados foram rotulados.

O conjunto escolhido foi o *Face Expression Recognition Dataset* (OHEIX, 2018). Este conjunto possui dados referentes a sete principais emoções do ser humano, e um total de 28821 arquivos, sendo: Raiva, Desgosto, Medo, Felicidade, neutro, triste e surpresa.

Inicia-se então a etapa de pré-processamento, onde é realizado uma formatação dos dados de entrada, para que todos sigam um mesmo padrão de dados. Durante esta etapa foram realizados ajustes nas dimensões (altura e largura)

das imagens, alteração da paleta de cores presente nas imagens para uma paleta de tons de cinza, para que todas apresentem o mesmo padrão e não causem interferência no treinamento da rede neural.

Com a conclusão desta etapa de padronização, seguiu-se para o processo de geração de massas, dividindo a massa recém processada e formatada em dois grupos, um de validação e outro de treinamento. A massa de validação serve como meio de validar os resultados após a inicial interação da rede sob a massa de treinamento, com o objetivo de quantificar os resultados obtidos da primeira interação com o conjunto de dados.

Figura 21 - Criação de massas de treinamento e validação.

```
train_dat = IDG()
val_dat = IDG()
batch_size = 128
epochs = 120

train_gen = train_dat.flow_from_directory(b_path+"train",target_size=(pic_size,pic_size)
                                          ,color_mode="grayscale",batch_size=batch_size,
                                          class_mode="categorical",shuffle=True)
val_gen = val_dat.flow_from_directory(b_path+"validation",target_size=(pic_size,pic_size),
                                      color_mode="grayscale",batch_size=batch_size,
                                      class_mode="categorical",shuffle=False)
```

Fonte: Autoria própria.

3.4 Configuração da rede neural

Fechada a etapa de pré-processamento e geração de massas, prosseguiu-se a configuração da rede neural. Foi iniciada pela camada de entrada de dados, onde foi configurado o formato dos dados que serão consumidos pela rede. Foi utilizada a função *Conv2D* para criar uma camada com um núcleo de convolução que recebe a massa de dados como entrada, e como saída gera um tensor de saídas capaz de identificar padrões nas entradas, possibilitando a rede extrair abstrações do dado.

Figura 22 - Criação da camada de convolução para entrada de dados.

```
model.add(Conv2D(64,(3,3),padding='same',input_shape=(48,48,1)))
```

Fonte: Autoria própria.

Com a entrada de dados configurada, foi adicionada uma camada de normalização dos dados, onde é feita uma transformação para manter a saída média próxima de 0 e o desvio padrão da saída próximo de 1.

Figura 23 - Criação da camada de normalização.

```
model.add(BatchNormalization())
```

Fonte: Autoria própria.

Foi criado em seguida uma camada com a função de ativação desejada, utilizando a função *Activation*. Posteriormente adicionou-se uma camada responsável por realizar o agrupamento máximo dos dados com a função *MaxPooling2D* e, por fim, adicionado uma camada de validação do treinamento, com o objetivo de evitar que a rede faça sobreajustes e cause impacto em sua performance e resultados, utilizando a função *Dropout*.

Figura 24 - Criação das camadas auxiliares da entrada.

```
model.add(Activation('tanh'))
model.add(MaxPooling2D(pool_size=(2,2)))
model.add(Dropout(0.25))
```

Fonte: Autoria própria.

Desta forma, foi criado uma entrada de dados para nossa rede e atingiu-se as seguintes linhas de código:

Figura 25 - Código referente a entrada de dados.

```
model = Sequential()
model.add(Conv2D(64,(3,3),padding='same',input_shape=(48,48,1)))
model.add(BatchNormalization())
model.add(Activation('tanh'))
model.add(MaxPooling2D(pool_size=(2,2)))
model.add(Dropout(0.25))
```

Fonte: Autoria própria.

Utilizando desta entrada de dados, foram criadas novas camadas com núcleos de convolução, com o objetivo de identificar diferentes padrões e possíveis combinações entre si.

Figura 26 - Adicionais camadas de convolução.

```
#layer 1
model = Sequential()
model.add(Conv2D(64,(3,3),padding='same',input_shape=(48,48,1)))
model.add(BatchNormalization())
model.add(Activation('tanh'))
model.add(MaxPooling2D(pool_size=(2,2)))
model.add(Dropout(0.25))
#layer 2
model.add(Conv2D(128,(5,5),padding='same'))
model.add(BatchNormalization())
model.add(Activation('tanh'))
model.add(MaxPooling2D(pool_size=(2,2)))
model.add(Dropout(0.25))
#layer 3
model.add(Conv2D(512,(3,3),padding='same'))
model.add(BatchNormalization())
model.add(Activation('tanh'))
model.add(MaxPooling2D(pool_size=(2,2)))
model.add(Dropout(0.25))
```

Fonte: Autoria própria.

Com as camadas de entrada de dados criadas, iniciou-se a configuração das camadas densas da rede neural, que possuem uma conexão profunda, ou seja, cada neurônio em uma camada densa recebe uma entrada de todos os neurônios da camada anterior. Esta camada foi criada utilizando a função *Dense*. Foi adicionada também uma camada de normalização, uma camada com a função de ativação e uma camada de validação, chegando as seguintes linhas de código:

Figura 27 - Criação da camada densa.

```
model.add(Dense(256))
model.add(BatchNormalization())
model.add(Activation('tanh'))
model.add(Dropout(0.25))
```

Fonte: Autoria própria.

Foi adicionada a primeira camada densa da nossa rede neural, utilizando deste padrão, foram replicadas alterando o número dimensões passadas na função *Dense*, atingido as seguintes linhas de código:

Figura 28 - Adição de novas camadas densas.

```
#FC Layer 1
model.add(Dense(256))
model.add(BatchNormalization())
model.add(Activation('tanh'))
model.add(Dropout(0.25))
#FC Layer 2
model.add(Dense(512))
model.add(BatchNormalization())
model.add(Activation('tanh'))
model.add(Dropout(0.25))
#FC Layer 3
model.add(Dense(1024))
model.add(BatchNormalization())
model.add(Activation('tanh'))
model.add(Dropout(0.25))
```

Fonte: Autoria própria.

Na última camada, foi adicionada uma camada densa com o número de emoções que buscou-se identificar. Foi atribuída a ela a função de ativação *Softmax*, responsável por calcular a distribuição de probabilidades do evento em N classes diferentes, ou seja, ela estabelece a porcentagem de cada classe. Foi adicionada esta camada utilizando a função *Dense*, e passando como parâmetro o número de classes e a função de ativação, resultando na seguinte linha de código.

Figura 29 - Adição da camada *Softmax*.

```
model.add(Dense(n_classes,activation='softmax'))
```

Fonte: Autoria própria.

Finalizando a configuração das camadas da nossa rede neural, seguimos para a seleção de nosso otimizador, utilizado para minimizar a nossa função de perda durante o treinamento. Com o otimizador selecionado, devemos definir a nossa função de perda também. No caso, a escolhida foi a *"categorical_crossentropy"* por existirem mais de dois rótulos possíveis, e pelo fato de precisarmos calcular a possibilidade de cada um dos rótulos para a entrada X, chegando às seguintes linhas.

Figura 30 - Compilando a configuração da rede neural.

```
opt = RMSprop(learning_rate = 0.0001)
model.compile(optimizer=opt,loss='categorical_crossentropy',metrics=['accuracy'])
```

Fonte: Autoria própria.

Encerrada a configuração da camadas, alcançando as seguintes linhas de código:

Figura 31 - Configuração da rede neural.

```
n_classes = 7
#layer 1
model = Sequential()
model.add(Conv2D(64,(3,3),padding='same',input_shape=(48,48,1)))
model.add(BatchNormalization())
model.add(Activation('tanh'))
model.add(MaxPooling2D(pool_size=(2,2)))
model.add(Dropout(0.25))
#layer 2
model.add(Conv2D(128,(5,5),padding='same'))
model.add(BatchNormalization())
model.add(Activation('tanh'))
model.add(MaxPooling2D(pool_size=(2,2)))
model.add(Dropout(0.25))
#layer 3
model.add(Conv2D(512,(3,3),padding='same'))
model.add(BatchNormalization())
model.add(Activation('tanh'))
model.add(MaxPooling2D(pool_size=(2,2)))
model.add(Dropout(0.25))

model.add(Flatten())
#FC Layer 1
model.add(Dense(256))
model.add(BatchNormalization())
model.add(Activation('tanh'))
model.add(Dropout(0.25))
#FC Layer 2
model.add(Dense(512))
model.add(BatchNormalization())
model.add(Activation('tanh'))
model.add(Dropout(0.25))
#FC Layer 3
model.add(Dense(1024))
model.add(BatchNormalization())
model.add(Activation('tanh'))
model.add(Dropout(0.25))

model.add(Dense(n_classes,activation='softmax'))
opt = RMSprop(learning_rate = 0.0001)
model.compile(optimizer=opt,loss='categorical_crossentropy',metrics=['accuracy'])
```

Fonte: Autoria própria.

3.5 Treinamento da rede neural

Para executarmos nossa rede neural com as camadas recém configuradas, foi utilizada a função *fit_generator*, passando um conjunto de parâmetros, como está sendo mostrado na imagem abaixo.

Figura 32 - Executando o treinamento da rede neural.

```
checkpoint = ModelCheckpoint("model_weights.h5",monitor = "val_acc",verbose=1,
                             save_best_only=True,mode = "max")

callbacks_list = [checkpoint]

history = model.fit_generator(generator=train_gen,
                              steps_per_epoch=train_gen.n//train_gen.batch_size,
                              epochs=epochs,
                              validation_data = val_gen,
                              validation_steps=val_gen.n//val_gen.batch_size,
                              callbacks = callbacks_list)
```

Fonte: Autoria própria.

Para cada parâmetro temos:

- *Generator*: Recebe a massa de treinamento recém criada.
- *steps_per_epoch*: Quociente do tamanho da massa de treinamento pelo *batch_size* da massa.
- *Epochs*: Número de iterações desejadas sob a massa de treinamento.
- *validation_data*: Recebe a massa de validação recém criada.
- *validation_steps*: Quociente do tamanho da massa de validação pelo *batch_size* da massa.
- *callback*: Recebe uma lista de funções de retropropagação que serão aplicadas durante o treinamento do modelo.

Com o fim da execução deste comando, temos criado um modelo treinado com base nos parâmetros passados na função e nas configurações definidas da rede neural. Por fim, foi salvo o modelo utilizando o seguinte comando:

Figura 33 - Salvando o modelo resultante do treinamento.

```
model.save("model_RMSprop_L3_BS64_E120.h5")
```

Fonte: Autoria própria.

Com a intenção de atingir a maior acurácia possível, foi realizada uma série de treinamentos variando funções de ativação, otimizadores, número de camadas densas, *batch size e epochs*. Iniciamos definindo as funções de ativação que foram escolhidas para os testes, elas foram Sigmóide, Tanh e *ReLU*, por possuírem grande uso treinamentos de redes neurais. Com o auxílio do professor orientador foram definidos os otimizadores utilizados nos treinamentos, eles são *Adagrad*, *Adam*, RMSProp e SGD. Neste, também, foi definido a variação de *batch size* que utilizamos nos treinamentos em 64 e 128, o número de *epochs* ou iterações sobre a massa de dados de treinamentos em 24, 48 e 60, e o número de camadas densas que testamos, foi decidido testar com duas e três.

Os treinamentos foram realizados utilizando a mesma massa de dados disponibilizada pelo *Face Expression Recognition Dataset* (OHEIX, 2018). Para tal variação foram realizadas mudanças no código na etapa de geração de massas, alterando os valores das variáveis *batch size* e *epochs*, de acordo com o treinamento que foi realizado.

Figura 34 - Variação batch size e número de epochs.

```
train_dat = IDG()
val_dat = IDG()
batch_size = 64
epochs = 24
train_gen = train_dat.flow_from_directory(b_path+"train",target_size=(pic_size,pic_size)
                                          ,color_mode="grayscale",batch_size=batch_size,
                                          class_mode="categorical",shuffle=True)
val_gen = val_dat.flow_from_directory(b_path+"validation",target_size=(pic_size,pic_size),
                                      color_mode="grayscale",batch_size=batch_size,
                                      class_mode="categorical",shuffle=False)
```

Fonte: Autoria própria.

A variação da função de ativação foi atingida ao alterar o parâmetro passado para a função *Activation*, variando entre *“relu”*, *“tanh”* ou *“sigmoid”* em todas as camadas de adicionadas que utilizaram esta função.

Figura 35 - Variação da função de ativação.

```
model.add(Activation('tanh'))
model.add(Activation('relu'))
model.add(Activation('sigmoid'))
```

Fonte: Autoria própria.

O número de camadas diferentes foi alcançado com a retirada de uma das camadas densas ou com a adição de uma. A alteração dos otimizadores foi realizada alterando a variável *"opt"* e atribuindo a ela o otimizador desejado no treino, como demonstra a imagem abaixo.

Figura 36 - Variação de otimizadores.

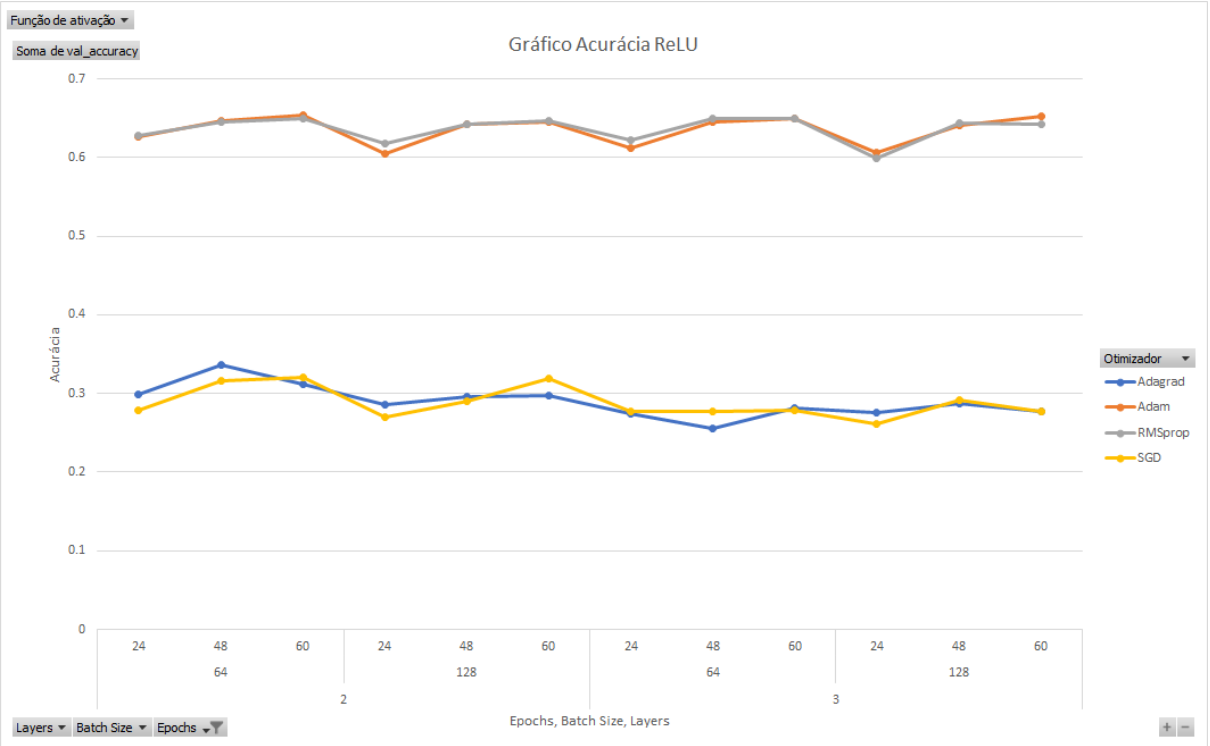
```
opt = Adagrad(learning_rate = 0.0001)
opt = Adam(learning_rate = 0.0001)
opt = RMSprop(learning_rate = 0.0001)
opt = SGD(learning_rate = 0.0001)
```

Fonte: Autoria própria.

3.6 Resultados

Como resultante da estratégia de variar funções de ativação, otimizadores, camadas densas, *batch size* e *epochs* para atingir a maior acurácia possível, foi realizado o treinamento de 144 modelos. Destes foram obtidos resultados extremamente positivos ao utilizarmos a função de ativação *ReLU*.

Figura 37 - Acurácia dos modelos utilizando ReLU.

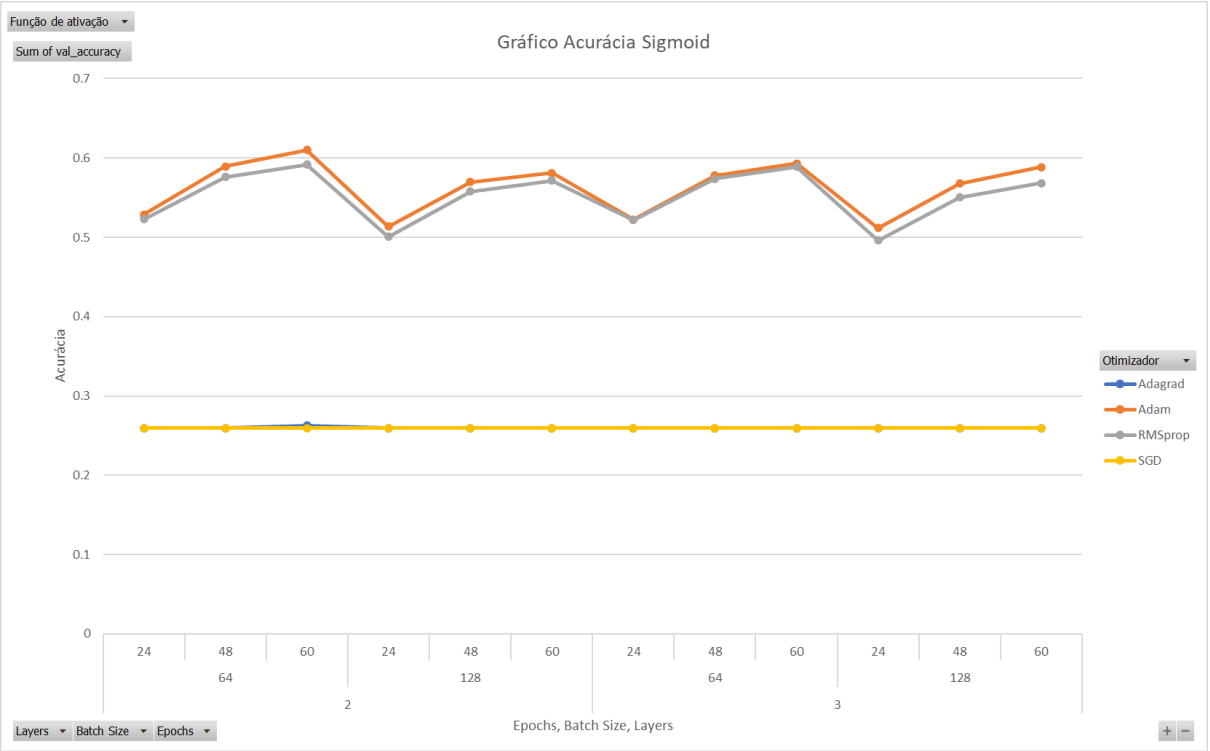


Fonte: Autoria própria.

Com o gráfico acima foi observado que os modelos que utilizaram como otimizadores o *Adam* e o RMSProp tiveram os melhores resultados, atingindo em alguns pontos da linha quase duas vezes a acurácia daqueles que utilizaram como otimizadores o *Adagrad* e o SGD. Com *ReLU* foi alcançada a acurácia máxima de de 65,34% ao utilizar o otimizador *Adam* com duas camadas densas, *batch size* de tamanho 64 e 60 *epochs*.

Observou-se que nos resultados dos modelos que utilizaram a função de ativação Sigmóide, apresentaram o mesmo comportamento das que utilizaram a função *ReLU*, onde em certos pontos do gráfico atingiram quase o dobro de acurácia ao utilizar os otimizadores *Adam* e RMSProp.

Figura 38 - Acurácia dos modelos utilizando Sigmóide.

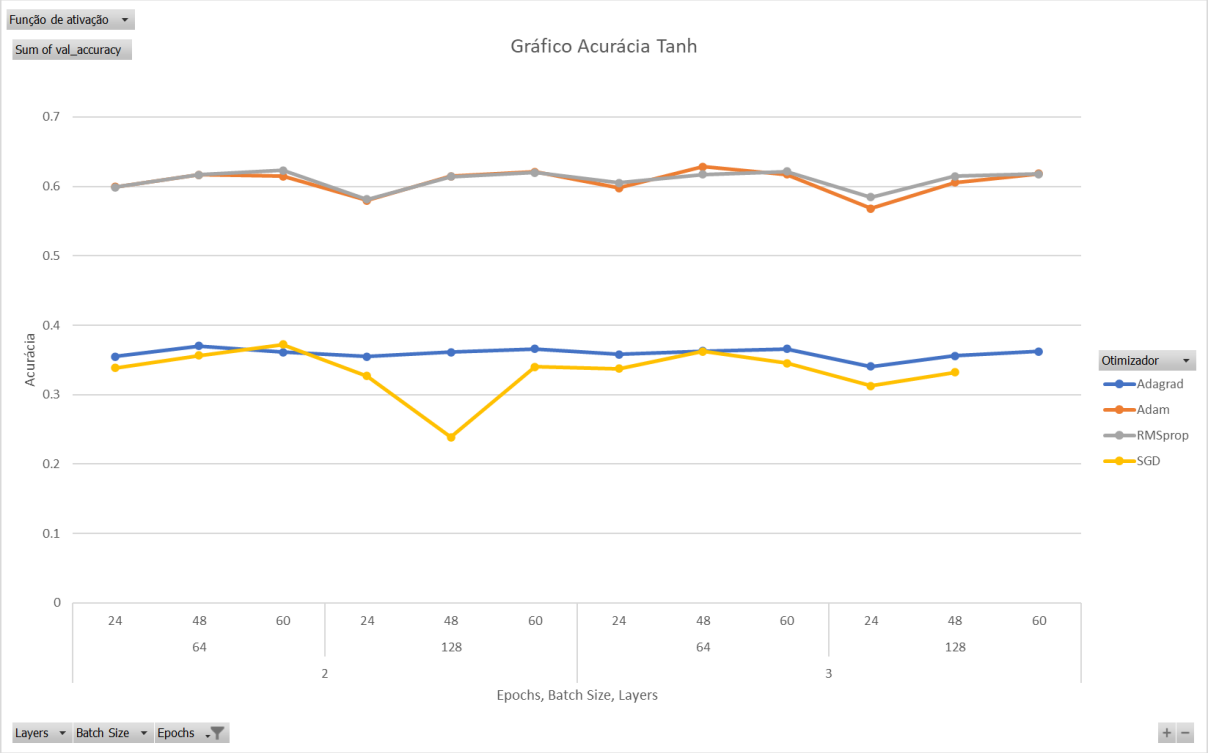


Fonte: Autoria própria.

Foi alcançada a acurácia máxima de 61,01% ao utilizar a função Sigmóide, com o otimizador *Adam* com duas camadas densas, *batch size* de tamanho 64 e 60 *epochs*. Constatou-se, também, que a utilização dos otimizadores *Adagrad* e SGD com a função Sigmóide afeta a performance do treinamento do modelo, chegando a acurácia máxima de 26,24%.

Com a função Tanh, também, foi encontrada uma grande diferença nos resultados ao utilizar *Adagrad* e SGD, sendo observado assim como nas outras funções em alguns pontos do gráfico quase o dobro da acurácia quando comparada aos que utilizaram *Adam* e RMSProp.

Figura 39 - Acurácia dos modelos utilizando Tanh.

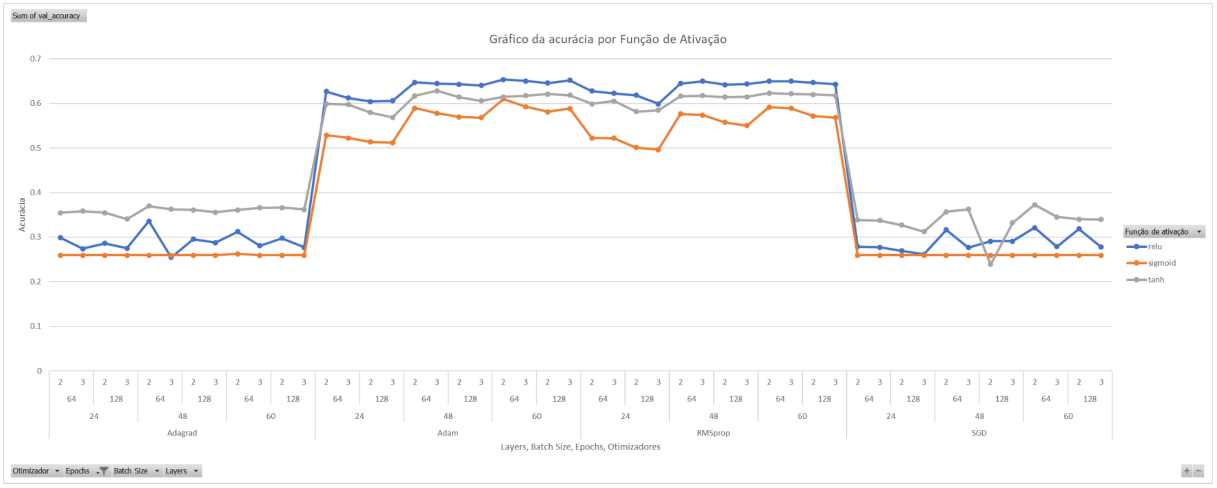


Fonte: Autoria própria.

Foi atingida a acurácia máxima de 62,81% com a função Tanh, utilizando do otimizador *Adam* com três camadas densas, *batch size* de tamanho 64 e 48 *epochs*. Estas similaridades encontradas com as funções Sigmóide e *ReLU* nos resultados que utilizaram os otimizadores *Adagrad* e SGD, podem sugerir uma ineficiência destes otimizadores para tal tarefa ou configurações utilizadas.

Ao ser analisado o gráfico abaixo, foi observado que as maiores acurácias foram atingidas ao utilizar a função de ativação *ReLU*. Também, foi constatado que os modelos que utilizaram dos otimizadores *Adam* e RMSProp, obtiveram os maiores valores de acurácia.

Figura 40 - Acurácia por função de ativação.



Fonte: Autoria própria.

4 Conclusão

Este trabalho foi concebido com o objetivo de realizar um estudo sobre a adaptação e a criação de um sistema inteligente capaz de realizar a detecção facial de um vídeo, e posteriormente determinar a emoção da pessoa retratada.

O trabalho foi segregado nas seguintes etapas, teve início com estudos referentes à inteligência artificial, quais as principais técnicas, suas aplicações e ferramentas possíveis. Seguido da escolha da técnica utilizada, no caso, foi redes neurais profundas ou *Deep Learning*. Iniciou-se com o treinamento da rede neural, inicialmente definindo a massa de dados que seria utilizada, e realizando o pré-processamento dela para uma padronização dos dados. Com os dados preparados foi iniciado o treinamento em si dos modelos, ocorrendo a coleta dos resultados apresentados nos modelos. Com o encerramento da etapa de treinamento dos modelos, foi realizada uma análise dos dados ou resultados obtidos no processo.

Com este trabalho chegamos a conclusão de que é possível usar a computação para detectar a emoção de uma pessoa por meio de suas expressões faciais. A solução que foi proposta, usando a massa de dados *Face Expression Recognition Dataset* (OHEIX, 2018) com uma base de 28821 imagens, chegou a uma acurácia de 65,34% utilizando a função de ativação *ReLU* e otimizador *Adam*. Quando considerado os resultados obtidos por meio deste trabalho, é indicado a utilização da função de ativação *ReLU* e do otimizador *Adam* para o treinamento de modelos relacionados ao reconhecimento de emoções por meio de imagens.

Observamos que utilizando outras combinações de otimizadores e funções de ativação o resultado foi menor, quando utilizados os otimizadores *Adagrad* e SGD. Obtidos resultados em alguns casos de 50 a 60% menores que a maior acurácia alcançada, sugeriu-se que estes otimizadores foram ineficientes para tal tarefa ou configurações utilizadas.

Foi possível criar um sistema de detecção simples e de forma desacoplada, de forma que no futuro podemos mudar as funções de ativação e os métodos de detecção facial para poder evoluir o sistema. Contanto, é bom realizar uma reflexão sobre o uso de tecnologias de reconhecimento facial, há questões éticas e legais que devem ser consideradas de forma cuidadosa.

O principal ponto a ser considerado é a questão da privacidade, o uso do reconhecimento facial para detectar pessoas sem o seu consentimento, pode ser considerada uma violação à privacidade e aos direitos do indivíduo. Além disso, existe o risco de que estas informações coletadas sejam utilizadas de forma maliciosa.

Outro ponto de atenção no uso desse tipo de tecnologia é que como ela é baseada em algoritmos de treinamento com dados coletados de determinados indivíduos, isso pode levar a um problema de precisão e equidade. Isso poderia resultar em alguns problemas ao tentar reconhecer o rosto de pessoas de certa raça ou etnia, ou seja, é sempre importante fazer um teste que englobe todos os tipos possíveis de usuários.

Além disso, é necessário garantir que essas tecnologias sejam feitas de forma que qualquer usuário tenha o direito de controlar a forma em que suas imagens são utilizadas. É fundamental que as empresas e organizações sejam transparentes e trabalhem com seus usuários para garantir que as preocupações éticas e legais sejam abordadas de maneira adequada.

4.1 Trabalhos Futuros

No período em que desenvolvemos o trabalho foram observados possíveis pontos de melhoria que poderiam ser implementados para um aprimoramento do projeto final. Porém como o tempo foi limitado, não conseguimos seguir com o desenvolvimento. As propostas para uma evolução desse trabalho são as seguintes:

- Testar outras formas de reconhecimento de expressão facial.
- Adaptar para usar em casos especiais, como por exemplo, aulas EAD para verificar o sentimento dos alunos durante o conteúdo da aula, sessão de psicologia com o objetivo de verificar o que o paciente sente durante a consulta, entre outros usos.
- Testar com outros conjuntos de dados para identificar se há um padrão na acurácia.

- Testar com outras arquiteturas de redes neurais, funções de ativação e otimizadores, com o fim de validar melhorias ou alterações nos resultados obtidos, buscando valores maiores para a acurácia.
- Gerar gráficos e relatórios mostrando a variação de emoção do usuário durante a execução do vídeo.
- Realizar análises e comparação da acurácia atingida por cada emoção.

5 Referências

ACTIVATION FUNCTIONS IN NEURAL NETWORKS. International Journal of Engineering Applied Sciences and Technology, Http://www.ijeast.com/, ano 2455-2143, v. 4, n. 12, p. 310-316, 2022.

BISSI, T. D. Reconhecimento facial com os algoritmos eigenfaces e fisherfaces. 2013.

BRAGA, L. F. Z. Sistemas de reconhecimento facial. 2013.

BOCHIE, Kaylane; et al. Aprendizado Profundo em Redes Desafiadoras: Conceitos e Aplicações. 2020.

CANTO, V. H. B. Uso de redes neurais convolucionais para identificação biométrica de indivíduos em tempo real utilizando internet das coisas. 2010.

Computer Vision Talks. Viola-Jones in Nut-shell. 2019. Disponível em: <http://computervisionwithvaibhav.blogspot.com/2015/08/viola-jones-in-nut-shell.html >. Acesso em: 23 de maio de 2019.

D. Jakhar, I. Kaur, Artificial intelligence, machine learning and deep learning: definitions and differences, *Clinical and Experimental Dermatology*, Volume 45, Issue 1, 1 January 2020, Pages 131–132, https://doi.org/10.1111/ced.14029

DINIZ, F. A. Redface – um sistema de reconhecimento de expressões faciais para apoiar um ambiente virtual de aprendizagem. 2013.

EKMAN, P. Darwin and facial expression: a century of research in review. New York: Academic Press, 1973. capo 4, p. 1 69-221 : Cross-cultural studies of facial expression.

EKMAN & FRIESEN, W.V. Origem, uso y codificación: bases para cinco categorías de conduta no verbal. In: VERÓN , E. et al. Lenguaje e comunicación social. Buenos Aires: Nueva Visión, 1971 . p. 51 -1 05.

FAJARDO, F. T. Desenvolvimento de uma aplicação web com detecção facial. 2019.

HECK; KÜNAS; PADOIN. Análise de Desempenho e Efetividade de Redes Neurais Convolucionais em Plataformas de GPU e CPU Aplicadas ao Reconhecimento de Emoções Através de Expressões Faciais em Seres Humanos. 2020.

ISHII, F. T. Lucas-Kanade Tensorial: Aplicação da álgebra tensorial para a estimativa de fluxo ótico em sequência de imagens coloridas. 2018.

KARLIK, B; OLGAC, A. V. Performance Analysis of Various Activation Functions in

Generalized MLP Architectures of Neural Networks. [2011?]

LEÃO, L. P.; BEZERRA, J. S.; MATOS, L. N.; NUNES, M. A. S. N. Detecção de expressões faciais: Uma abordagem baseada em análise do fluxo óptico. 2012.

LIBRALON, G. L. Modelagem computacional para reconhecimento de emoções baseada na análise facial. 2014.

LOPES, M. C. C. F. A importância da publicidade emocional no marketing viral. 2015.

MACEDO, A. L. F; GOMES, I. R. Algoritmo Haar Cascade Aplicado na Detecção das Placas de Parada Obrigatória e de Velocidade Máxima Permitida. 2020.

MUNIZ, R. S; RODRIGUES, R. G; GUEDES, G. P. Cromoterapia e computação afetiva: auxiliando os estados de ansiedade. 2016.

OHEIX, JONATHAN. Face expression recognition dataset. In: OHEIX, JONATHAN. Face expression recognition dataset. [S. l.], 2018. Disponível em: https://www.kaggle.com/datasets/jonathanoheix/face-expression-recognition-dataset. Acesso em: 2 ago. 2022.

OLIVEIRA, A. Como agências de publicidade podem usar pesquisa de mercado? Disponível em: <https://mindminers.com/blog/como-agencias-de-publicidade-podem-usar-pesquisa-de-mercado/>. Acesso em 25 de novembro de 2022.

OLIVEIRA, E.; JAQUES, P. A. Inferindo as emoções do usuário pela face através de um sistema psicológico de codificação facial. 2008.

OMAIA, D. e. a. Um sistema para detecção e reconhecimento de face em vídeo utilizando a transformada cosseno discreta. 2009.

PAIXÃO, E. P. Aplicação do algoritmo Viola-Jones na detecção de objetos. 2018.

PATIL, R.; SAHULA, V. Automatic recognition of facial expressions in image sequences: A review. 2010.

P. Ongsulee, "Artificial intelligence, machine learning and deep learning," 2017 15th International Conference on ICT and Knowledge Engineering (ICT&KE), Bangkok, Thailand, 2017, pp. 1-6, doi: 10.1109/ICTKE.2017.8259629.

RAO, K. S; KOOLAGUDI, S. G. Recognition of emotions from video using acoustic and facial features. 2013.

RIBEIRO, M; GUIMARÃES, S. Redes Neurais utilizando TensorFlow e Keras. 2018.

RIBEIRO, A. M; JUNIOR, F. P. S. A. Um Estudo Comparativo Entre Cinco Métodos de Otimização Aplicados Em Uma RNC Voltada ao Diagnóstico do Glaucoma. 2020.

RODRIGUES, J. Pesquisa indica recursos mais relevantes de mídias sociais + 95 estatísticas de redes em 2022. Disponível em: <https://resultadosdigitais.com.br/marketing/estatisticas-redes-sociais/#:~:text=Estat%C3%ADsticas%20gerais%20de%20Redes%20Sociais,usu%C3%A1rios%20de%202021%20para%202022> Acesso em 25 de novembro de 2022.

ROSEIRO, L. F. W; COSTA, V. K. S. Reconhecimento facial como solução para automatização da marcação de presença. 2020.

SANTOS, M. E. M.; PINTO, Y. K. S. Emotion Recognition. [*S. l.*], 30 nov. 2022. Disponível em: https://github.com/MathEiji/emotion_recognition. Acesso em: 30 nov. 2022.

SHIMABUKURO, S. J; SOUSA, V. Detecção de pedestres em cruzamentos utilizando características Haar. 2011.

SILVA, A. A. N. Reconhecimento de peças do jogo progster utilizando aprendizado profundo. 2019.

SILVA, J. A.; SILVA, M. J. P. Expressões Faciais e Emoções Humanas - Levantamento Bibliográfico. 1995.

SILVA, J. B. Explorando o algoritmo de Viola-Jones na detecção e reconhecimento facial. 2018.

TensorFlow. Repositório do TensorFlow no GitHub. Disponível em: <https://github.com/tensorflow/tfjs-models/tree/master/face-detection>. Acesso em 25 de novembro de 2022.

TORRE, F. De la T.; COHN, J. F. Facial Expression Analysis. Springer, Https://doi.org/10.1007/978-0-85729-997-0_19, p. 377–409, 2011.

TRINDADE, L. L. Detecção de obstáculos para carros autônomos utilizando aprendizado profundo. 2018.

ZHANG, K. et al. Joint face detection and alignment using multitask cascaded convolutional networks. IEEE Signal Processing Letters, v. 23, n. 10, p. 1499–1503, 2016. Disponível em: <https://arxiv.org/pdf/1604.02878.pdf>. Acesso em 07 de junho de 2020.